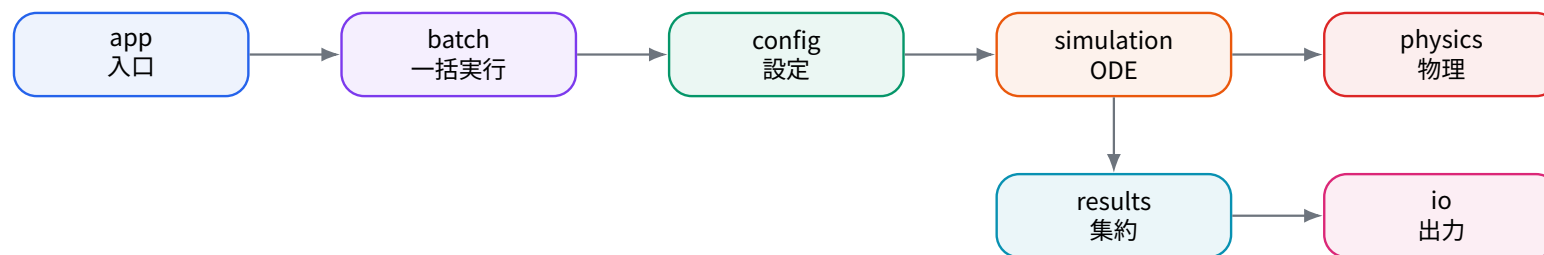


PROGRAM STRUCTURE GUIDE

ハイブリッドロケット燃焼シミュレータ プログラム木構造ガイド

最新プログラムを初めて読む人のための実装地図



本資料は理論ガイドではなく、プログラムの枝構造、責務、呼び出し関係、ファイル単位の読み方を示す地図である。説明は木構造に合わせて、全体 -> 大枝 -> 下位枝 -> ファイルへ潜る形で構成する。

読み方: 木構造に沿ってフラクタルに潜る

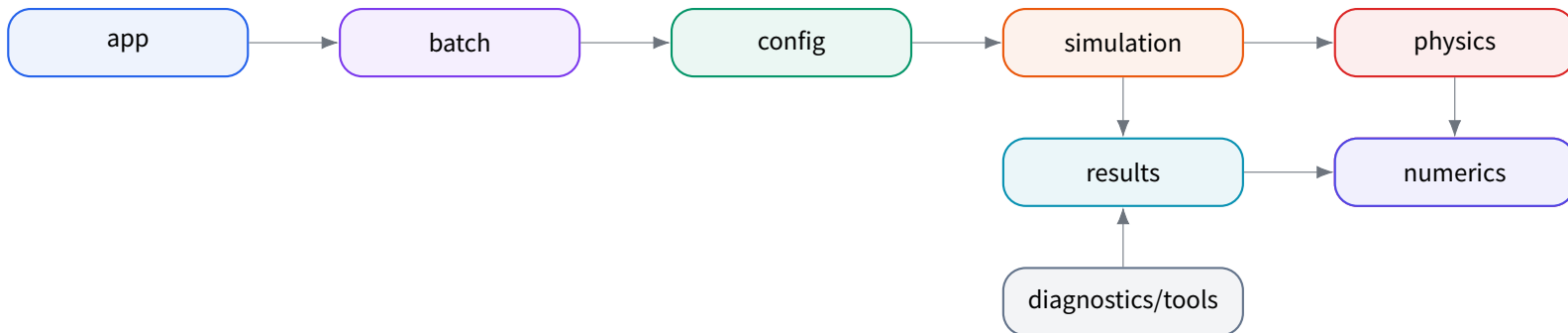
この文書は、章ごとに第一層の枝を扱い、その章の中で第二層の枝へ潜り、必要なら第三層、第四層の枝へさらに降りる。各枝では同じ型を繰り返す。

枝ページで見るもの	意味
責務	その枝がプログラム全体の中で何を守るために存在するか。
読むもの / 渡すもの	呼び出し時に受け取る情報と、下流または上流へ返す情報。
連携先	縦方向の依存と横方向のデータ受け渡し。
内部木構造	その枝の一段下を図として見る。必要なら次ページ以降でさらに潜る。
ファイル責務	もう潜る必要がない粒度のファイルを、役割と主要 API で確認する。

同じ説明が繰り返されるのは意図的である。大枝で把握し、下位枝で同じ型を再利用して詳細へ降りることで、巨大なプログラムを一度に記憶しなくてよい構造にしている。

全体木構造

最初にこのページで全体を把握する。大きな流れは、入口・一括実行・設定構築・ODE 結合・物理モデル・結果生成・ファイル出力である。



枝	大まかな責務	主な連携先
app	run.py から呼ばれ、通常実行か一括実行かを判定し、適切なアプリケーション処理へ処理を流す。	config, batch, simulation, results, io
batch	CSV 各行を設計ケースとして解釈し、設定反映、並列実行、進捗表示、batch_summary 作成までを統括する。	config, app.model_sweep, io.excel
config	JSON 設定を読み、単位と値を検証し、物理コンポーネントと Engine を構築する。	app, batch, physics.components
simulation	タンク、インジェクタ、燃焼室、ノズルを一時刻ごとに評価し、状態ベクトルを時間発展させる。	physics.components, results
physics	materials, flow, components に分け、材料物性・流れ構成式・実体部品を混ぜない。	simulation.rhs
results	solution と event log から時系列表、summary、保存用データを作る。	io
io	Excel, JSON, text, filesystem 操作を物理モデルから切り離す。	app, batch, results
numerics	補間、root solve、最適化を物理モデルから切り離して提供する。	physics.materials, flow
diagnostics	本計算ではなく、入力データ・表補間・比較グラフの検査を行う。	tools
tools	本体ロジックを diagnostics へ委譲し、ユーザーが単独実行しやすい入口だけを持つ。	diagnostics

app

階層1: 実行入口を束ねる枝

run.py から呼ばれ、通常実行か一括実行かを判定し、適切なアプリケーション処理へ処理を流す。

読むもの	プロジェクトルート、0_Settings、0_Batch_Execution の有無	渡すもの	model sweep の結果、または batch 実行結果
連携先	config, batch, simulation, results, io	代表フロー	run.py -> app.cli -> app.model_sweep / batch.execution.executor

この枝の内部木構造

```
app/  
├── sweep/  
├── cli.py  
└── model_sweep.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
app/sweep/	一つの設定セットに対し、指定されたインジェクタモデルを順にまたは並列に実行し、共通出力へ集約する。
cli.py	run.py から呼ばれる最上位 dispatcher。batch 検出と通常 sweep の分岐だけを担う。
model_sweep.py	通常実行のモデルスイープ全体を統括する。SPI/HEM/Dyer の結果を共通出力へまとめる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
cli.py	run.py から呼ばれる最上位 dispatcher。batch 検出と通常 sweep の分岐だけを担う。	main()	run.py 実行時
model_sweep.py	通常実行のモデルスイープ全体を統括する。SPI/HEM/Dyer の結果を共通出力へまとめる。	run_injector_model_sweep()	0_Batch_Execution が ないとき

app/sweep

階層2: SPI/HEM/Dyer の通常モデルスイープ

一つの設定セットに対し、指定されたインジェクタモデルを順にまたは並列に実行し、共通出力へ集約する。

読むもの	設定辞書、run_models、thermo table 状態	渡すもの	モデル別フォルダ、model_summary、比較グラフ
連携先	simulation.Engine, io, diagnostics	代表フロー	config -> worker -> Engine.solve -> results/io

この枝の内部木構造

```
app/sweep/  
├── config.py  
├── execution.py  
├── io.py  
├── status.py  
└── worker.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
config.py	sweep 実行時のモデル選択、並列設定、thermo prewarm 条件を解釈する。
execution.py	逐次/並列に model worker を動かし、結果を回収する。
io.py	sweep 出力フォルダ、settings copy、summary 出力をまとめる。
status.py	モデル worker の status JSON ペイロードと progress callback を作る。
worker.py	1 つのインジェクタモデルを実行する worker 単位。Engine の solve と保存を呼ぶ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
config.py	sweep 実行時のモデル選択、並列設定、thermo prewarm 条件を解釈する。	_set_json_value(), _dedupe_preserve_order(), _compact_none(), _prefixed_filename(), _planned_injector_models() ...	model_sweep 開始時
execution.py	逐次/並列に model worker を動かし、結果を回収する。	model_parallel_settings(), run_models_parallel(), run_models_sequential()	model_sweep 実行中
io.py	sweep 出力フォルダ、settings copy、summary 出力をまとめる。	Tee, model_sweep_output_root(), prepare_model_sweep_dirs(), write_sweep_common_outputs()	sweep 結果保存
status.py	モデル worker の status JSON ペイロードと progress callback を作る。	_write_status(), _design_status_payload(), _make_design_status_callback()	並列 sweep の進捗表示

ファイル	責務	主な API	使われる時
worker.py	1 つのインジェクタモデルを実行する worker 単位。Engine の solve と保存を呼ぶ。	_run_single_model_worker(), _normalize_model_record()	SPI/HEM/Dyer 各モデル実行

batch

階層1: 一括入力 CSV から複数設計を実行する枝

CSV 各行を設計ケースとして解釈し、設定反映、並列実行、進捗表示、batch_summary 作成までを統括する。

読むもの	0_Batch_Execution, Batch_Execution_Config.json, CSV	渡すもの	design 別結果、batch_summary.xlsx, input_mapping, batch_errors
連携先	config, app.model_sweep, io.excel	代表フロー	config -> input -> execution -> output -> io.excel.batch_summary

この枝の内部木構造

```
batch/
├── config/
├── console/
├── execution/
├── input/
├── output/
├── planning/
├── constants.py
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
batch/config/	batch 設定ファイルを読み、CSV 名、出力先、入力仕様の前提を整える。
batch/console/	計算本体から独立して、準備 activity と計算 progress を読みやすく表示する。
batch/execution/	各設計を worker へ渡し、並列または逐次に行き、進捗・結果・エラーを回収する。
batch/input/	Input_Specification と Input_Design_Name を解釈し、CSV の値を JSON 設定ツリーへ反映する。
batch/output/	モデル別 summary や入力ステータスから、batch_summary とエラーレポート用データを作る。
batch/planning/	batch 中で thermo table を事前生成できるか、各設計で作るべきかを判断する。
constants.py	batch 内で共有する定数名、status 名などを持つ。
model.py	DesignCase や BatchResult など batch 用データ構造を定義する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
constants.py	batch 内で共有する定数名、status 名などを持つ。	-	batch 全域
model.py	DesignCase や BatchResult など batch 用データ構造を定義する。	InputColumnSpec, MonitorSpec, RowExecutionResult, BatchExecutionError	batch 全域

batch/config

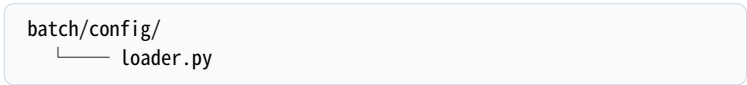
階層2: 一括実行設定の読み込み

batch 設定ファイルを読み、CSV 名、出力先、入力仕様の前提を整える。

読むもの	Batch_Execution_Config.json, 0_Batch_Execution	渡すもの	batch 実行用の設定辞書とパス情報
連携先	batch.input, batch.execution	代表フロー	config -> input -> execution -> output -> io.excel.batch_summary

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
loader.py	Batch_Execution_Config.json を読み、CSV パスと Copy_Batch_Execution を扱う。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
loader.py	Batch_Execution_Config.json を読み、CSV パスと Copy_Batch_Execution を扱う。	_read_json(), _batch_csv_base_name(), detect_batch_execution(), _copy_batch_execution_dir()	batch 準備

batch/input

階層2: CSV 行を設計ケースへ変換する枝

Input_Specification と Input_Design_Name を解釈し、CSV の値を JSON 設定ツリーへ反映する。

読むもの	CSV DataFrame, Input_Specification, design name 列	渡すもの	設計 ID、設計名、反映済み設定、入力セルステータス
連携先	config.units, config.validation	代表フロー	CSV row -> unit conversion -> settings override -> validation cell status

この枝の内部木構造

```
batch/input/
├── design_name.py
├── destinations.py
├── row_config.py
└── specs.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
design_name.py	Input_Design_Name 列から設計名を読み、ファイル名安全な表記へ sanitize する。
destinations.py	Input_Specification の宛先 JSON パスを解決する。
row_config.py	CSV 1 行を設定辞書へ反映し、InputError をセル位置へ戻す。
specs.py	Input_Specification, Input_Design_Name, Monitored_Values を読み取る。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
design_name.py	Input_Design_Name 列から設計名を読み、ファイル名安全な表記へ sanitize する。	<code>_sanitize_design_name()</code> , <code>_design_label()</code> , <code>_display_design_label()</code>	CSV 読み込み後
destinations.py	Input_Specification の宛先 JSON パスを解決する。	<code>_get_destination_node()</code> , <code>_get_destination_unit()</code> , <code>_set_destination_value()</code> , <code>_normalized_destination()</code> , <code>_columns_for_destinations()</code>	CSV 値反映時
row_config.py	CSV 1 行を設定辞書へ反映し、InputError をセル位置へ戻す。	<code>_prepare_row_config()</code> , <code>_infer_input_validation_rejections()</code>	各 Design 準備

ファイル	責務	主な API	使われる時
specs.py	Input_Specification, Input_Design_Name, Monitored_Values を読み取る。	_collect_input_specs(), _collect_design_name_column(), _design_names_from_csv(), _design_labels_from_names(), _collect_monitor_specs()	batch 準備

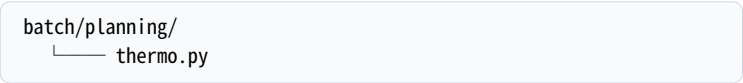
batch/planning

階層2: 実行前の計画と thermo 準備判断

batch 中で thermo table を事前生成できるか、各設計で作るべきかを判断する。

読むもの	入力仕様と CEA signature に関わる設定	渡すもの	prewarm 可否、実行計画
連携先	batch.execution, chamber thermochemistry	代表フロー	config -> input -> execution -> output -> io.excel.batch_summary

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
thermo.py	batch 入力が thermo signature を変え得るかを見て prewarm 可否を判断する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
thermo.py	batch 入力が thermo signature を変え得るかを見て prewarm 可否を判断する。	_batch_overrides_thermo_table_signature()	batch 準備

batch/execution

階層2: 設計ケースの実行管理

各設計を worker へ渡し、並列または逐次に実行し、進捗・結果・エラーを回収する。

読むもの	設計ケース、ベース設定、実行モード	渡すもの	設計別 summary、worker status、error rows
連携先	app.model_sweep, batch.console, batch.output	代表フロー	preparation -> worker_count -> parallel/sequential -> progress_monitor -> artifacts

この枝の内部木構造

```
batch/execution/  
├── executor.py  
├── layout.py  
├── parallel.py  
├── preparation.py  
├── progress_monitor.py  
├── results.py  
├── sequential.py  
├── worker.py  
└── worker_count.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
executor.py	batch 全体の高レベル入口。準備、実行、出力の各枝を順に呼ぶ。
layout.py	設計 ID/モデル別の出力フォルダ構造を決める。
parallel.py	複数設計を ProcessPool 等で並列実行する。
preparation.py	batch 出力 root、入力 CSV、設計名、入力仕様など実行前準備をまとめる。
progress_monitor.py	parallel/sequential 実行中に worker status を読んで console へ渡す。
results.py	worker から戻る成功/失敗結果の形を整える。
sequential.py	設計を一つずつ順番に実行する。デバッグ時に有用。
worker.py	1 設計ケースを実行する worker 本体。入力エラー時の扱いもここで分岐する。
worker_count.py	利用する worker 数を設計数・設定値から決める。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
executor.py	batch 全体の高レベル入口。準備、実行、出力の各枝を順に呼ぶ。	run_batch_execution()	run_batch_execution
layout.py	設計 ID/モデル別の出力フォルダ構造を決める。	write_settings_dir_from_config(), set_json_value(), prepare_worker_config(), ensure_design_layout()	worker 開始前
parallel.py	複数設計を ProcessPool 等で並列実行する。	_parallel_failure_result(), _run_parallel_designs()	design_parallel 有効時
preparation.py	batch 出力 root、入力 CSV、設計名、入力仕様など実行前準備をまとめる。	prepare_batch_result_root(), print_batch_header(), prewarm_or_explain_thermo_cache()	batch 開始直後

ファイル	責務	主な API	使われる時
progress_monitor.py	parallel/sequential 実行中に worker status を読んで console へ渡す。	progress_line_for_pending()	batch 実行中
results.py	worker から戻る成功/失敗結果の形を整える。	input_error_result(), sweep_result_to_row(), run_error_result()	worker 完了時
sequential.py	設計を一つずつ順番に実行する。デバッグ時に有用。	_run_sequential_designs()	design_parallel 無効時
worker.py	1 設計ケースを実行する worker 本体。入力エラー時の扱いもここで分岐する。	_execute_design(), _execute_design_worker()	各 Design worker
worker_count.py	利用する worker 数を設計数・設定値から決める。	_select_design_worker_count()	parallel 実行前

batch/console

階層2: 一括実行中の表示を担う枝

計算本体から独立して、準備 activity と計算 progress を読みやすく表示する。

読むもの	worker status JSON、進捗単位	渡すもの	コンソール表示文字列
連携先	batch.execution.progress_monitor	代表フロー	worker status JSON -> activity/progress_units -> bars -> console line

この枝の内部木構造

```
batch/console/  
├── activity.py  
├── bars.py  
├── progress_units.py  
└── status_io.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
activity.py	準備フェーズだけに表示する activity のマイルストーンを決める。25/25 到達保証もここに関わる。
bars.py	activity 用のドットバー、progress 用のシャープバーなど文字バーの整形だけを担う。
progress_units.py	設計内の SPI/HEM/Dyer 進捗を batch 全体の進捗単位へ換算する。
status_io.py	worker status JSON を読み書きする。表示側と worker 側の共有ファイル境界。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
activity.py	準備フェーズだけに表示する activity のマイルストーンを決める。25/25 到達保証もここに関わる。	preparation_activity_count(), activity_milestone_counts(), activity_milestone_lines()	progress が動き始める前
bars.py	activity 用のドットバー、progress 用のシャープバーなど文字バーの整形だけを担う。	format_progress_bar(), format_activity_bar()	console 表示生成
progress_units.py	設計内の SPI/HEM/Dyer 進捗を batch 全体の進捗単位へ換算する。	design_progress_units()	progress percent 計算
status_io.py	worker status JSON を読み書きする。表示側と worker 側の共有ファイル境界。	write_status_file(), read_status_snapshot()	worker status 更新/監視

この枝は、親の batch から見ると「表示」に見えるが、内部では読み書き、進捗単位化、activity 判定、バー整形が分離されている。したがって、console 表示を変えるときは、何を変えるのかを先に決める。状態ファイル形式なら status_io、進捗率なら progress_units、表示タイミングなら activity、見た目なら bars を見る。

batch/output

階層2: batch 成果物の組み立て

モデル別 summary や入力ステータスから、batch_summary とエラーレポート用データを作る。

読むもの	各 worker 結果、入力マッピング、エラー情報	渡すもの	batch_summary 用行、batch_errors、input_mapping
連携先	io.excel.batch_summary, io.text	代表フロー	worker results -> model rows/error reports -> batch_summary writer

この枝の内部木構造

```
batch/output/  
├── artifacts.py  
├── model_rows.py  
└── reports.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
artifacts.py	batch 全体の成果物保存をまとめて呼ぶ。
model_rows.py	各モデル summary から batch_summary シート行を作る。
reports.py	batch_errors.json/txt や input_mapping の保存を担当する。

直接ファイルの責務と主な API

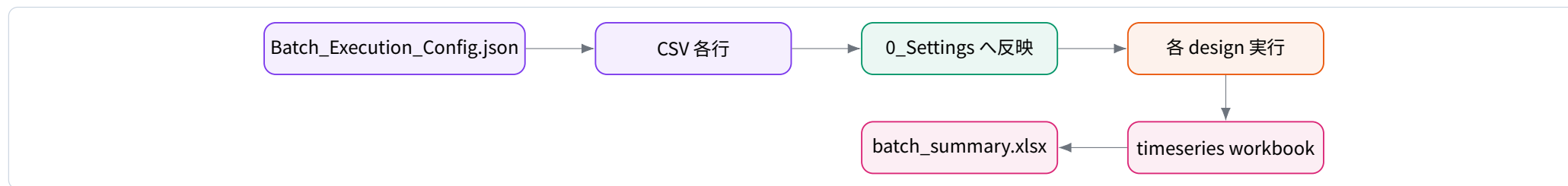
ファイル	責務	主な API	使われる時
artifacts.py	batch 全体の成果物保存をまとめて呼ぶ。	write_batch_outputs(), cleanup_worker_status_files()	batch 終了時
model_rows.py	各モデル summary から batch_summary シート行を作る。	build_model_sheet_rows()	batch summary 作成前
reports.py	batch_errors.json/txt や input_mapping の保存を担当する。	_write_input_mapping_text(), _write_design_error_files(), _write_batch_errors()	batch 終了時

0_Batch_Execution

入力 CSV から複数設計を作るユーザー設定枝

ここは src/hybrid_motor_sim/batch/ の実装ではなく、ユーザーが直接編集する 0_Batch_Execution/ の仕様である。通常実行では読まれず、0_Batch_Execution/Batch_Execution_Config.json と指定 CSV が存在すると一括実行モードに入る。

読むもの	Batch_Execution_Config.json, 指定 CSV	渡すもの	design ごとの上書き済み settings、design name、monitor 指定
主な連携	batch/config, batch/input, batch/execution, io/excel/batch_summary	代表フロー	JSON 仕様読取 -> CSV 列検証 -> 単位変換 -> 設定反映 -> 各 design 実行 -> workbook 監視値抽出



内部木構造とファイル

```
0_Batch_Execution/  
├── Batch_Execution_Config.json  
└── <File_Name>.csv
```

要素	役割	プログラムへの影響
File_Name.Value	入力 CSV 名。拡張子は指定せず、.csv として解決する。	batch/config/loader.py が CSV パスを決める。
Input_Specification.Value	CSV 列名ごとに、入力単位と反映先 Destination を定義する。	batch/input/specs.py が列を検証し、batch/input/row_config.py が行ごとの settings を作る。
Input_Design_Name.Value	任意の CSV 列を design 名として使う。null なら数値 ID のみ。	design フォルダ名・console 表示・batch summary の Name 列に反映される。
Monitored_Values.Value	batch_summary へ抜き出す workbook シート、列、単位、集計位置を定義する。	io/excel/batch_summary/monitoring.py が timeseries workbook から値を抽出する。

Batch_Execution_Config.json/Input_Specification

CSV 列を 0_Settings へ接続する仕様

Input_Specification.Value は「CSV 列名 -> 反映先設定値」の対応表である。CSV 列名は完全一致で照合される。列名の末尾に [mm] や [MPa] が含まれていても、それは人間に読みやすい名前であり、実際の変換には JSON 側の Unit が使われる。

読むもの	CSV 列名、CSV セル値、Unit、Destination	渡すもの	変換済みの settings dict
主な連携	config/units, batch/input/destinations.py	代表フロー	CSV 値 -> 単位変換 -> Destination の Value を書き換え



```
"ノズルスロート径 d_t [mm]": {
  "Unit": "[mm]",
  "Destination": "7_Nozzle/d_t"
}
```

この指定は、File_Name で指定した入力 CSV から ノズルスロート径 d_t [mm] に完全一致する列を探し、その列の各セル値を [mm] として解釈し、0_Settings/7_Nozzle.json の d_t/Value を書き換える、という意味である。書き換え時には、CSV 側の Unit から、Destination 先に書かれている Unit へ自動変換される。

キー	意味	実装上の扱い
CSV 列名	"ノズルスロート径 d_t [mm]" のような辞書キー。	入力 CSV の列名と完全一致で照合する。列がなければ InputError。
Unit	CSV セル値をどの単位として読むか。	config/units が Destination 側 Unit へ変換する。
Destination	書き換える設定値の場所。	7_Nozzle/d_t は 0_Settings/7_Nozzle.json の d_t/Value を意味する。

Destination の規則。 7_Nozzle/d_t のように、ファイル番号名から始まり、JSON 内の枝を / で降る。Destination の最後が Value/Unit/Definition/Note 形式の設定項目なら、その Value だけを書き換える。存在しない枝、Unit を持たない枝、空欄セルは InputError として扱われる。

Batch_Execution_Config.json/Input_Design_Name と CSV 仕様

設計 ID に人間が読める名前を添える

CSV の各行は自動的に Design ID = 1, 2, ... として扱われる。Input_Design_Name.Value に CSV 列名を指定すると、その列の値が design 名として読み込まれ、結果フォルダや console 表示では <ID>_<Name> 形式になる。一方、batch_summary.xlsx では ID 列は数値だけを保持し、Name 列に設計名を分離して表示する。

項目	仕様	注意
CSV encoding	pandas で読み込める CSV。現在の同梱例は BOM 付き UTF-8 でも読める。	列名は完全一致。全角・半角・空白の違いに注意する。
行の意味	1 行が 1 設計。全行に対して SPI/HEM/Dyer 等の指定モデルを実行する。	途中の行だけ失敗しても batch 全体は可能な限り継続する。
Design ID	CSV の行順に 1 から自動採番する。	batch summary の第一列は常に数値 ID。
Design Name	任意列。危険文字、空白、長すぎる文字列はファイル名安全化される。	<:"/\ ?*> と制御文字は _ に置換される。
入力セルの色	受理済みは薄青、Design Name は薄紫、InputError は薄赤。	cross-field validation で落ちた値も対応セルが薄赤になる。

```
"Input_Design_Name": {  
  "Value": "Design_Name"  
}
```

Value が null なら、設計名機能は無効になり、結果フォルダや console 表示は数値 ID だけを使う。列名が指定されている場合は、入力 CSV からその列を完全一致で探し、各行の値をファイル名として安全な文字列へ整形したうえで <ID>_<Name> の表示に使う。

Batch_Execution_Config.json/Monitored_Values

batch_summary へ抜き出す値の仕様

Monitored_Values.Value は、各設計・各モデルの時系列 Excel から、比較したい値を batch_summary.xlsx へ集約する仕様である。Source は Sheet/Header 形式で、時系列 workbook 内のシート名とヘッダー名を完全一致で参照する。Location は「どのフェーズの、どの統計量か」を表す。

読むもの	timeseries workbook のシート名・列名、Unit、Location	渡すもの	batch summary のモデル別シート列
主な連携	results/timeseries/workbook_schema.py, io/excel/batch_summary/monitoring.py	代表フロー	Sheet/Header -> phase filter -> statistic -> unit conversion -> cell value

```
" 特性排気速度 [m/s]": {
  "Unit": "[m/s]",
  "Source": "Results/c_star[m/s]",
  "Location": "average@liquid"
}

" 燃料ピッチ慣性 [kg*m^2]": {
  "Unit": "[kg*m^2]",
  "Source": "For_Mechanics/I_pitch_fuel[kg*m^2]",
  "Location": "last@all"
}
```

この指定は、batch_summary.xlsx に指定ラベルの列を追加し、各設計・各モデルの時系列 Excel から Results/c_star[m/s] や For_Mechanics/I_pitch_fuel[kg*m^2] を読み、指定 Unit へ単位変換したうえで、average@liquid などの操作により値を算出する、という意味である。Source の / は最初の 1 個だけをシート名と列名の区切りとして扱うため、kg/m^2/s のように列名側に / を含む単位も使用できる。

構成要素	選択肢	意味
Source	Results/<header>, For_Mechanics/<header>	timeseries workbook のシート名とヘッダー名を完全一致で指定する。シート名を省略した指定は無効である。
統計操作	first, last, average, max, min	first/last は対象範囲の端点を取る。average/max/min は数値化できない値、NaN、非有限値を除外してから計算する。
フェーズ範囲	all, liquid, gas	all は全時系列、liquid は tank_phase が液相共存、gas は気相ブローダウンの行だけを使う。
Location	<operation>@<phase>	使用可能な指定は (first,last,average,max,min) と (all,liquid,gas) の直積すべてである。

探索リストを別に持たない。 監視値抽出は、実際に生成された timeseries workbook のシート名とヘッダー名に対して完全一致で行う。シート名は results/timeseries/workbook_schema.py の RESULTS_SHEET_NAME と MECHANICS_SHEET_NAME が単一ソースであり、抽出側が独自の列名リストを持つことはない。Source 不一致や抽出不能な指定は、計算そのものの失敗ではなく output_error として扱われる。

config

階層1: 設定ロード・検証・モデル構築の枝

JSON 設定を読み、単位と値を検証し、物理コンポーネントと Engine を構築する。

読むもの	0_Settings/*.json	渡すもの	Engine, run_models, runtime configuration
連携先	app, batch, physics.components	代表フロー	loading -> validation -> factory -> Engine

この枝の内部木構造

```
config/
├── factory/
├── loading/
├── units/
└── validation/
```

下位枝・直接ファイルの役割

名前	この階層での識別
config/factory/	Oxidizer/Tank/Injector/Chamber/Nozzle/Engine を生成する。物理式は持たず依存注入を行う。
config/loading/	プロジェクトルート、設定ディレクトリ、JSON 設定群を確定する。
config/units/	mm, MPa, %, L などの表記を SI 内部値へ変換し、空欄・null・単位不一致も扱う。
config/validation/	幾何寸法、充填率、効率、モデル選択など、ユーザー入力として危険な値を検出する。

config/loading

階層2: 設定ファイルとパス解決

プロジェクトルート、設定ディレクトリ、JSON 設定群を確定する。

読むもの	project_root, settings_dir	渡すもの	settings dict, resolved paths
連携先	config.factory	代表フロー	loading -> validation -> factory -> Engine

この枝の内部木構造

```
config/loading/  
├── project.py  
└── settings.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
project.py	プロジェクトルートと settings_dir の解決を行う。
settings.py	0_Settings 配下の JSON を読み込む。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
project.py	プロジェクトルートと settings_dir の解決を行う。	find_project_root(), resolve_settings_dir()	起動時
settings.py	0_Settings 配下の JSON を読み込む。	read_json(), load_settings()	config build 前

config/validation

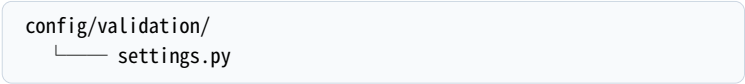
階層2: 設定値の妥当性検証

幾何寸法、充填率、効率、モデル選択、設定 JSON の書式など、ユーザー入力として危険な値を検出する。

読むもの	settings dict	渡すもの	Validation error または通過結果
連携先	batch.input, config.factory	代表フロー	loading -> validation -> factory -> Engine

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
settings.py	幾何・充填率・効率・モデル名に加え、Value と並列な JSON キーの許可範囲、Options の有無、リスト選択式の値を検証する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
settings.py	0_Information を除く settings 項目で、Value, Options, Unit, Definition, Note* 以外の並列キーを禁止し、選択式項目の Options と入力値を検証する。	validate_config_values()	component 生成前

config/units

階層2: CSV 入力と JSON 設定をつなぐ単位変換

mm, MPa, %, L などの表記を SI 内部値へ変換し、空欄・null・単位不一致も扱う。

読むもの	CSV セル値、Unit 文字列	渡すもの	SI 値、変換済み設定値、セルエラー
連携先	batch.input.row_config	代表フロー	loading -> validation -> factory -> Engine

この枝の内部木構造

```
config/units/  
├── blank.py  
├── conversion.py  
├── converters.py  
├── errors.py  
├── labels.py  
├── parser.py  
├── registry.py  
└── value.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
blank.py	空欄、null、未指定の扱いを一箇所に集約する。
conversion.py	実際の単位変換計算を行う。
converters.py	unit conversion の public facade。batch 側の import 面を短く保つ。
errors.py	単位変換エラー型とメッセージを扱う。
labels.py	単位ラベルの正規化と表示名を扱う。
parser.py	Unit 文字列やセル表現を解析する。
registry.py	対応する単位ラベルと SI 係数を登録する。
value.py	値と単位の組を扱う小さな構造を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
blank.py	空欄、null、未指定の扱いを一箇所に集約する。	is_blank()	CSV セル処理
conversion.py	実際の単位変換計算を行う。	conversion_spec(), convert_value()	CSV 値反映時
converters.py	unit conversion の public facade。batch 側の import 面を短く保つ。	-	CSV 値変換
errors.py	単位変換エラー型とメッセージを扱う。	UnitConversionError	変換失敗時
labels.py	単位ラベルの正規化と表示名を扱う。	extract_unit_from_label()	変換前処理
parser.py	Unit 文字列やセル表現を解析する。	_Parser, _dims_to_tuple(), _combine(), _power(), normalize_unit() ...	CSV 読み込み時
registry.py	対応する単位ラベルと SI 係数を登録する。	-	変換時

ファイル	責務	主な API	使われる時
value.py	値と単位の組を扱う小さな構造を提供する。	ParsedUnit, ConversionSpec, parse_feet_inches_value()	変換時

config/units 仕様

自動単位変換の文法・対応範囲・限界

自動単位変換は、一括実行 CSV の入力値と 0_Settings の Unit を接続するための補助機能である。単発実行の 0_Settings/*.json を丸ごと SI へ変換する機能ではない。

読むもの	CSV 側 Unit、Destination 側 Unit、CSV セル値	渡すもの	Destination の Value へ入れる変換済み値
主な連携	batch/input/specs.py, batch/input/row_config.py	代表フロー	parse unit -> dimension check -> factor/offset -> Value へ代入

単位式の文法

表記	例	説明
角括弧付き単位	[mm], [MPa], [kg/(m ² *s)]	CSV/JSON の Unit 欄で使う基本形。
乗算・除算	[kg/(m ² *s)], [N*s]	*, /, 括弧を使える。
整数冪	[m ²], [m ³]	^2, ^3 のような整数冪に対応。
無次元	[-]	次元なし。次元付き単位とは変換不可。
未定義	...	ファイル名やモデル名など、単位変換しない値。
シンボリック	[(m/s)/(kg/(m ² *s))^n]	^n を含む単位は同一表記のみ許可。後退速度係数用。

対応単位と接頭語

分類	対応単位	備考
接頭語	m, c, h, k, M	それぞれ 1e-3, 1e-2, 1e2, 1e3, 1e6 として解釈する。接頭語は、コード内で prefix 可と登録された基底単位にだけ付けられる。
長さ	m, in, ft と prefix 付き長さ	mm, cm, km などは m に接頭語が付いたものとして扱う。
質量	kg, g, t, lb と prefix 付き g, t	kg は基底単位として扱い、さらに kkg のような接頭語は付けない。
時間	s, min, h	s は SI 単位だが接頭語を付けない。h は時間単位として扱い、接頭語 h (hecto) とはトークンとして区別して処理する。
力	N, kgf, lbf と prefix 付き N	N は力カテゴリであり、kg*m/s ² へ自動簡約しない。
圧力	Pa, bar, atm, PSI, psi と prefix 付き Pa, bar	MPa, kPa, mbar などは接頭語として処理する。
体積	m ³ , L, cc と prefix 付き m, L	cm ³ は (cm) ³ として解釈される。mL も L の接頭語付きとして扱う。
物質質量	mol と prefix 付き mol	kmol, mmol などに対応する。
温度	K, °C と prefix 付き K	°C <-> K の 273.15 offset 変換は単独温度のときだけ行う。

分類	対応単位	備考
割合	%, vol%, mol%, wt%	%と各種割合単位の間は等倍変換可能。vol% <-> mol% など異種割合同士はエラー。

単位式の文法と制限

項目	仕様
演算子	積 *, 除算 /、整数べき ^、括弧に対応する。演算順序は通常の数式と同様に扱う。
接頭語の結合	cm^2 は (cm)^2 と解釈する。接頭語は単位名と強く結びつく。
次元判定	長さ、質量、時間、力、圧力、体積、物質質量、温度、割合などのカテゴリ指数が一致する場合だけ変換する。
SI への完全簡約なし	N と kg*m/s^2 は物理的には同じでも、この変換器では別カテゴリなので変換しない。登録された単位集合の範囲でだけ変換する。
温度 offset	単独の [°C] と [K] の相互変換だけ 273.15 を加減する。[°C/s] のような組立単位では温度差として扱い、offset 変換は行わない。
無次元と未定義	[-] は無次元量、... は単位を定義しない値である。両者は区別され、互いに変換しない。
後退係数 a	[(m/s)/(kg/(m^2*s))^n] のように ^n を含むシンボリック単位は同一表記のみ等倍で通す。a は n に依存するため、一般単位変換対象外である。
feet-inch	{ft}'{in} は特殊表記として認識されるが、batch 変換での一般出力単位としては使わない。

安全側の考え方。この単位変換器は、接頭語付き SI/一部 SI 併用単位と、実務で使う非 SI 単位を限定的に変換するための層である。物理式を使った換算、たとえば体積率から質量率への換算や、力を kg*m/s^2 に分解しての換算は行わない。変換不能な単位は InputError として扱い、暗黙の誤変換を防ぐ。

config/factory

階層2: 設定から実行可能オブジェクトを組み立てる枝

Oxidizer、FeedSystem、Chamber、Nozzle、Engine を生成する。物理式は持たず、設定値の単位変換・正規化済み値を component へ依存注入する。

読むもの	検証済み settings	渡すもの	Engine と各 component
連携先	physics.components, simulation.system	代表フロー	settings -> FeedSystem/Chamber/Nozzle -> Engine

config/factory/__init__.py は、settings accessor と loading/validation 系の軽い API を即時に公開し、build_engine_from_config() と build_engine_from_settings() は呼び出し時に builder.py を遅延 import する。これにより、NIST CSV schema check のように Engine を構築しない診断枝は、RocketCEA や物理 component 生成に依存せずに読み込める。

この枝の内部木構造

```
config/factory/
├── accessors.py
├── builder.py
├── chamber.py
├── engine.py
├── feed_system.py
├── models.py
└── nozzle.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
accessors.py	settings dict から必須値や section を安全に取り出す。
builder.py	settings 読み込みから Engine 構築までの public 入口。
chamber.py	CEA, Fuel, ChamberOxidizer, Chamber を組み立てる。
engine.py	component bundle を Engine へ渡す最終構築。
feed_system.py	Oxidizer, Valve, Injector を作り、FeedSystem facade へ結合する。
models.py	run_models や injector model 名の正規化を行う。
nozzle.py	Nozzle 幾何と環境圧を構築する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
accessors.py	settings dict から必須値や section を安全に取り出す。	unwrap_value(), require_section(), require_value(), require_cstar_efficiency(), safe_str() ...	factory 各所
builder.py	settings 読み込みから Engine 構築までの public 入口。	build_engine_from_config(), build_engine_from_settings()	app/batch から呼ばれる
chamber.py	CEA, Fuel, ChamberOxidizer, Chamber を組み立てる。	build_chamber()	Engine 構築
engine.py	component bundle を Engine へ渡す最終構築。	build_engine_shell()	build_engine_from_config
feed_system.py	Oxidizer, Valve, Injector, FeedSystem を組み立てる。	build_feed_system()	Engine 構築

ファイル	責務	主な API	使われる時
models.py	run_models や injector model 名の正規化を行う。	configured_run_models()	factory 開始時
nozzle.py	Nozzle 幾何と環境圧を構築する。	build_nozzle()	Engine 構築

0_Settings

単発実行と一括実行の基準になる JSON 設定枝

0_Settings/ は、単発実行でも一括実行でも最終的に使われる基準設定である。一括実行では CSV の各行がこの設定を複製し、Input_Specification の Destination だけを書き換える。つまり、CSV に載っていない値は 0_Settings/ の値がそのまま使われる。

読むもの	0_Settings/*.json	渡すもの	Engine, components, solver, output settings
主な連携	config/loading, config/validation, config/factory	代表フロー	JSON ロード -> 値検証 -> component 生成 -> simulation 実行

この枝の内部木構造

```
0_Settings/  
├── 0_Information.json  
├── 1_Oxidizer.json  
├── 2_Fuel.json  
├── 3_Tank.json  
├── 4_Valve.json  
├── 5_Injector.json  
├── 6_Chamber.json  
├── 7_Nozzle.json  
├── 8_Environment.json  
└── 9_Config.json
```

下位ファイルの役割

ファイル	主な責務
0_Information.json	設定対象名やメモを置く情報ファイル。入力値スキーマ検証の対象外。
1_Oxidizer.json	N2O 物性、PR-EOS、NASA9、RocketCEA 酸化剤名を定義する。
2_Fuel.json	fuel card、燃料後退係数、グレイン幾何を定義する。
3_Tank.json	タンク初期圧・体積・充填指定・任意のタンク内径を定義する。
4_Valve.json	任意の全開 feed valve を、Kv と代表内径、coupling 設定、診断モデルで定義する。
5_Injector.json	インジェクタ幾何、SPI/HEM/Dyer、気相 SPI/PR 設定を定義する。
6_Chamber.json	初期 O/F、燃焼室体積、c* 効率、thermo table を定義する。
7_Nozzle.json	スロート径、膨張比、ノズル分岐計算の幾何を定義する。
8_Environment.json	大気圧、環境温度など外部条件を定義する。
9_Config.json	solver、並列実行、出力名、thermo table 運用を定義する。

設定項目の共通形式

キー	意味	扱い
Value	実際にプログラムが読む値。	factory は基本的にこの値だけを取り出す。一括実行ではここが書き換わる。
Options	選択式項目で使用可能な値と、その値の意味。	文字列選択、null/number 選択、リスト選択式に置く。true/false 項目には付けない。
Unit	人間向け単位情報と一括実行時の変換先単位。	単発実行では自動変換されず、各 factory/component が期待単位で解釈する。一括実行では CSV Unit からこの Unit へ変換する。
Definition	設定の短い意味。	プログラム実行には使わない。ドキュメント・レビュー用。

キー	意味	扱い
Note, Note2, ...	注意点、適用範囲、設計上の意味。	プログラム実行には使わない。複数の注意は Note2, Note3 のように分ける。

書式制約。 0_Information.json を除き、Value と並列に置けるキーは Options, Unit, Definition, Note, Note2, ... だけである。source などの非公式メモ項目や、Value の中に混ざったメモ的要素は使わない。選択式項目では Options を Value の直後に置く。

0_Settings/1_Oxidizer.json

N2O 物性・CEA 酸化剤名・PR-EOS を定義する

酸化剤設定は、タンク飽和物性、気相 PR ブローダウン、インジェクタ流量、燃焼室 CEA 入力にまたがる。最も下層の物性データであり、ここを変更すると複数の物理枝へ波及する。

設定パス	Unit	内容	主な影響先
identity/Name	...	RocketCEA へ渡す酸化剤名。	chamber/thermochemistry/cea/backend.py
identity/MolW	[g/mol]	N2O モル質量。	タンクモル数、質量流量、燃焼室 O/F、積算質量。
thermophysical_data/Path_csv	...	NIST 飽和物性 CSV のパス。	materials/nitrous_oxide/saturation/table.py
thermophysical_data/T_min	[K]	低温側イベント/範囲ガードの基準。	タンク温度下限、物性表アクセス。
thermophysical_data/saturation_bounds_policy	...	飽和物性範囲外を clip または raise で扱う。	ODE 内部試行点、デバッグ時の異常検出。
peng_robinson/T_c, p_c, omega	[K], [Pa], [-]	PR-EOS の N2O 臨界物性。	気相タンク圧、PR 気相流量、departure 物性。
ideal_gas_heat_capacity/NASA9/temperature_policy	...	NASA9 係数の温度範囲外評価を extrapolate または clip で扱う。	PR 気相エネルギー式、PR 等エントロピー流量。
ideal_gas_heat_capacity/NASA9/low_temperature_interval/*	various	200–1000 K の NASA9 係数区間。	理想気体比熱、エンタルピー、エントロピー。
ideal_gas_heat_capacity/NASA9/high_temperature_interval/*	various	1000–6000 K の NASA9 係数区間。	理想気体比熱、エンタルピー、エントロピー。

NASA9 係数の構造。 各温度区間は $T_{\min}$, $T_{\max}$, coefficients を持つ。係数配列の出典や並び順は Note と Note2 に書き、source のような非公式キーは使わない。

0_Settings/2_Fuel.json

燃料カード・後退速度・グレイン幾何を定義する

燃料設定は、RocketCEA の fuel card、燃料後退式、燃料ポート体積、燃料残量の計算に使われる。燃料密度は fuel card 内の density_g_cc が燃料グレイン密度としても使われる。

設定パス	Unit	内容	主な影響先
cea/fuel_card	...	fuel 名、元素組成、生成エンタルピー、基準温度、密度。	RocketCEA 燃焼物性、燃焼温度、 c^* , Isp。
regression_model/a	$[(\text{m/s})/(\text{kg}/(\text{m}^2\cdot\text{s}))^n]$	後退速度係数。	fuel_grain/model.py の ddt_r、燃料流量。
regression_model/n	[-]	後退速度指数。	Go 依存性、O/F シフト、燃焼時間。
grain_geometry/L	[mm]	燃料長さ。	ポート体積、燃料流量、燃料質量。
grain_geometry/D	[mm]	燃料外径。	最大ポート半径、fuel_port_limit event、スライバ率。
grain_geometry/d_init	[mm]	初期ポート径。	初期 Go、燃料後退、初期燃料質量。

入力検証。 D>d_init は必須であり、違反すると config/validation/settings.py で InputError になる。一括実行では対応する CSV セルが薄赤になる。

0_Settings/3-5: Tank / Valve / Injector.json

供給系の初期状態・任意バルブ・流量モデルを決める

タンク、Valve、インジェクタは燃焼室へ入る酸化剤流量を支配するため、推力履歴へ最も直接的に効く枝である。Valve は無効なら供給系から外れ、有効なら液相時だけ全開 Kv 圧損要素として Tank と Injector の間に入る。

設定パス	Unit	内容	主な影響先
3_Tank/p_init	[MPa]	初期タンク圧。指定時は飽和表から初期温度を逆算する。	タンク初期温度、初期飽和圧、初期流量。
3_Tank/V	[cc]	タンク内部体積。	初期モル数、液相枯渇時刻、気相ブローダウン、For_Mechanics のタンク長さ導出。
3_Tank/Tank_geometry/d	[mm]	任意のタンク内径。null ならタンク CG・慣性列を出さない。	For_Mechanics の CG_tank, l_axis_tank, l_pitch_tank。
3_Tank/gas_liquid_ratio_Determinant/filling_rate	[vol\%]	液相体積充填率。	初期液相/気相モル数。
3_Tank/gas_liquid_ratio_Determinant/m_tot	[kg]	総酸化剤質量指定。	filling_rate の代替。ちょうど一方だけを指定する。
4_Valve/enabled	[-]	全開 Valve 圧損モデルを有効にする。	true なら液相時に Tank->Valve->Injector、false なら Tank->Injector。
4_Valve/Kv	[m ³ /h]	全開 Valve の水基準流量係数。	液相 Valve 圧損、p_valve_exit[MPa]、供給流量。
4_Valve/d_inner	[mm]	全開 Valve 流路の代表内径。	G_valve, u_valve, Void Fraction 診断。
4_Valve/coupling/mode	...	predictor または root。	Valve/Injector 連成の計算方法と計算時間。
4_Valve/diagnostics/void_fraction_models	...	出力するボイド率診断モデルのリスト。	Void_Fraction_HEM, Void_Fraction_Zivi, Void_Fraction_RA。
5_Injector/Cd	[-]	有効流量係数。	SPI/HEM/Dyer/gas flow の全流量に掛かる。
5_Injector/d_orifice, N_orifice	[mm], [-]	オリフィス径と孔数。	総オリフィス面積、酸化剤流量。
5_Injector/HEM_config/*	various	HEM 探索点、cache 刻み、臨界テーブル設定。	二相流量の精度と速度。
5_Injector/Gas_config/gas_blowdown_model	...	気相フェーズ流量を SPI または PR から選ぶ。	SPI はタンク気相密度の Bernoulli 型近似、PR は実在気体等エントロピー流れ。
5_Injector/run_models	...	通常実行で走らせる液相モデル群。	app model sweep、batch 内の各 design 実行。

充填指定と Valve 仕様。 filling_rate と m_tot は、どちらか一方だけを非 null にする。Valve は液相時に固定仕様の Kv_SPI 圧損要素として扱われ、気相ブローダウン時は pass-through で p_valve_exit=p_tank とする。

0_Settings/6_Chamber.json

燃焼室蓄積・CEA・thermo table を決める

燃焼室設定は、燃焼室体積、初期 O/F、c* 効率、RocketCEA direct/table 切替、thermo table 範囲と診断基準を持つ。

設定パス	Unit	内容	主な影響先
6_Chamber/V_pre,V_post	[cc]	プレ/ポストチャンバー体積。	燃焼室体積、圧力応答。
6_Chamber/OF_init	[-]	初期の微小 storage 正則化用 O/F。	立ち上がり時の数値安定性。
6_Chamber/Cstar_Efficiency	[-]	実効 c* 効率。	Nozzle.flow の有効温度、c*, 推力履歴。
6_Chamber/thermo/mode	...	table または cached_direct の熱物性取得。	ODE RHS 速度と補間誤差。
6_Chamber/thermo/dp_fd,dOF_fd	[MPa], [-]	CEA 偏微分の有限差分幅。	圧力 ODE の安定性。
6_Chamber/thermo_table/*	various	table 範囲、格子数、cache、診断、auto refine。	thermochemistry/table 全体。
6_Chamber/eval_bounds/*	various	ODE 評価中の p/O-F 安全範囲。	table/direct query のクリップと fallback。

0_Settings/7-9: Nozzle / Environment / Config.json

ノズル・環境・実行制御を決める

これらは物理部品の幾何と、計算をどの範囲・精度・出力先で実行するかを決める。

設定パス	Unit	内容	主な影響先
7_Nozzle/eps	[-]	膨張比 A_e/A_t 。	出口面積、分岐判定、圧力推力。
7_Nozzle/d_t	[mm]	スロート径。	スロート面積、チョーク流量、燃焼室圧。
8_Environment/p_amb	[Pa]	周囲圧。	ノズル背圧、推力圧力項、分岐判定。
8_Environment/T_amb	[K]	周囲温度。	p_init 未指定時のタンク初期温度など。
9_Config/tags/run_name	...	結果フォルダ名へ付ける run 名。	timestamp 付き result directory 名。
9_Config/tags/result_file_tag	...	timeseries/summary/event log ファイル名へ付ける任意 suffix。	結果ファイル名。
9_Config/solver/t_step	[s]	出力時系列の基本時間刻み。event 時刻はこの等間隔列に追加される。	Results, For_Mechanics の基本サンプリングと相境界行。
9_Config/solver/t_max	[s]	積分の最大時刻。	solve_ivp の上限と強制終了条件。
9_Config/solver/rtol	[-]	solve_ivp 相対許容誤差。	ODE 解精度と計算時間。
9_Config/solver/atol	[-]	solve_ivp 絶対許容誤差。	小さい状態量の誤差制御。
9_Config/solver/max_step	[s]	solve_ivp 内部最大刻み。null なら明示制限なし。	event 検出と計算時間。
9_Config/events/F_cutoff	[N]	推力 cutoff イベントしきい値。	thrust_cutoff 停止判定。
9_Config/events/liquid_eps	[mol]	液相枯渇判定の残液モルしきい値。	液相->気相 segment 切替。
9_Config/events/tank_temperature_margin	[K]	NIST 下限へ近づいた時の clamp 開始 margin。	タンク温度下限イベント。
9_Config/events/thrust_event_enable_time	[s]	推力 cutoff を有効にする最小時刻。	t=0 近傍の誤停止防止。
9_Config/progress/*	various	単発実行の進捗表示。	console progress。
9_Config/results/*	various	結果 root、設定コピー有無。	1_Results と保存処理。
9_Config/batch_run/*	various	batch/design 並列数と polling。	一括実行の速度と console 更新。

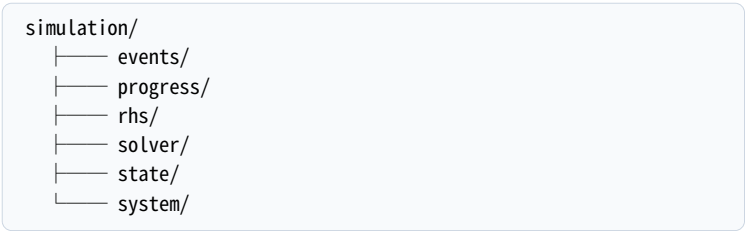
simulation

階層1: ODE システムとして部品を結合する枝

タンク、インジェクタ、燃焼室、ノズルを一時刻ごとに評価し、状態ベクトルを時間発展させる。

読むもの	Engine state, component models, solver config	渡すもの	solution, event log, solver stats
連携先	physics.components, results	代表フロー	Engine.solve -> RHS evaluation -> component calls -> solver segments -> results

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
simulation/events/	液相枯渇、温度下限、推力カットオフなどの event 関数を提供する。
simulation/progress/	一つのモデル実行中の進捗を軽量に表示する。batch の console とは別の枝。
simulation/rhs/	一時刻に必要なタンク圧、インジェクタ流量、CEA 物性、ノズル流量を評価し、状態微分へ変換する。
simulation/solver/	solve_ivp を呼び、event 後の状態整理、次 segment への切替、event 時刻を含む最終 solution 時刻列の組み立てを行う。
simulation/state/	ODE 配列の添字、初期状態、Jacobian sparsity を集中管理する。
simulation/system/	外部 API としての Engine を保持し、solve, summary, save_results を下位モジュールへ委譲する。

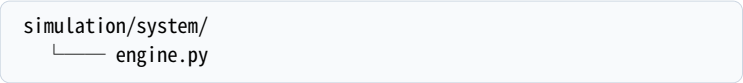
simulation/system

階層2: Engine facade

外部 API としての Engine を保持し、solve, summary, save_results を下位モジュールへ委譲する。

読むもの	component bundle, runtime config	渡すもの	solution and saved outputs
連携先	simulation.solver, results, io	代表フロー	Engine.solve -> RHS evaluation -> component calls -> solver segments -> results

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
engine.py	外部から見る Engine facade。solve や保存を下位モジュールへ委譲する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
engine.py	外部から見る Engine facade。solve や保存を下位モジュールへ委譲する。	Engine	app/model_sweep から

simulation/state

階層2: 状態ベクトルの意味と初期化

ODE 配列の添字、初期状態、Jacobian sparsity を集中管理する。

読むもの	component initial states	渡すもの	y0, indices, sparsity pattern
連携先	simulation.rhs, solver	代表フロー	Engine.solve -> RHS evaluation -> component calls -> solver segments -> results

この枝の内部木構造

```
simulation/state/  
├── indices.py  
├── initialization.py  
└── jacobian.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
indices.py	状態ベクトル配列の添字を一元管理する。
initialization.py	各 component から ODE 初期状態 y0 を作る。
jacobian.py	Radau 用の Jacobian sparsity 構造を作る。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
indices.py	状態ベクトル配列の添字を一元管理する。	-	RHS/solver 全域
initialization.py	各 component から ODE 初期状態 y0 を作る。	initial_engine_state()	solve 開始時
jacobian.py	Radau 用の Jacobian sparsity 構造を作る。	build_engine_jac_sparsity()	solve_ivp 呼び出し前

simulation/rhs

階層2: ODE 右辺の一時刻評価

一時刻に必要なタンク圧、インジェクタ流量、CEA 物性、ノズル流量を評価し、状態微分へ変換する。

読むもの	t, y, component models	渡すもの	dy/dt and step diagnostics
連携先	physics.components	代表フロー	state vector -> feed_system/chamber/nozzle -> derivatives

この枝の内部木構造

```
simulation/rhs/  
├── derivatives.py  
└── evaluation.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
derivatives.py	StepEvaluation を状態微分 dy/dt へ変換する。
evaluation.py	一時刻に必要な中間量を評価し、重複計算を防ぐ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
derivatives.py	StepEvaluation を状態微分 dy/dt へ変換する。	engine_derivative()	ODE RHS 内
evaluation.py	一時刻に必要な中間量を評価し、重複計算を防ぐ。	chamber_nozzle_flow()	ODE RHS 内

simulation/events

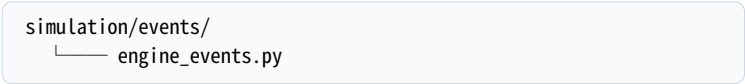
階層2: 相切替・停止条件

液相枯渇、温度下限、推力カットオフなどの event 関数を提供する。

読むもの	t, y, engine context	渡すもの	zero-crossing functions
連携先	simulation.solver	代表フロー	Engine.solve -> RHS evaluation -> component calls -> solver segments -> results

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
engine_events.py	液相枯渇、温度下限、推力カットオフ等の event 関数を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
engine_events.py	液相枯渇、温度下限、推力カットオフ等の event 関数を提供する。	current_phase(), thrust_margin(), liquid_remaining(), tank_temperature_margin_event(), fuel_port_margin_event() ...	solve_ivp events

simulation/solver

階層2: solve_ivp segment loop

solve_ivp を呼び、event 後の状態整理、次 segment への切替、event 時刻を含む最終 solution 時刻列の組み立てを行う。

読むもの	RHS, events, initial state	渡すもの	merged solution, event log
連携先	simulation.events, state	代表フロー	initial segment -> solve_ivp -> event handling -> next segment

この枝の内部木構造

```
simulation/solver/  
├── engine_runner.py  
├── event_handling.py  
├── segments.py  
└── state.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
engine_runner.py	solve_ivp segment loop 本体。event 発生ごとに状態を整える。
event_handling.py	event 後の phase 切替、clamp、停止判定を整理する。
segments.py	segment ごとの解を結合し、event 境界時刻では後続 segment の状態を優先して評価する補助。
state.py	solver 状態、統計、診断カウンタ初期化、event 時刻を含む最終 solution の組み立てを担当する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
engine_runner.py	solve_ivp segment loop 本体。event 発生ごとに状態を整える。	solve_engine()	Engine.solve
event_handling.py	event 後の phase 切替、clamp、停止判定を整理する。	handle_segment_event()	event 発生時
segments.py	segment ごとの解を結合し、event 境界時刻では後続 segment の状態を優先して評価する。	triggered_event_name(), termination_category(), evaluate_segments()	phase 切替時
state.py	solver 状態、統計、診断カウンタ初期化、event 時刻を含む最終 solution の組み立てを担当する。	reset_engine_solve_state(), prepare_initial_segment_state(), accumulate_solver_stats(), finalize_engine_solution()	solver 内部

simulation/progress

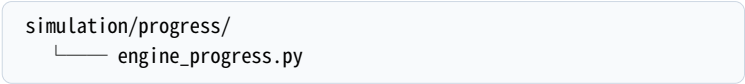
階層2: 単発実行の進捗表示

一つのモデル実行中の進捗を軽量に表示する。batch の console とは別の枝。

読むもの	time/state progress	渡すもの	console progress line
連携先	simulation.solver	代表フロー	Engine.solve -> RHS evaluation -> component calls -> solver segments -> results

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
engine_progress.py	単発 Engine 実行の progress callback を整形する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
engine_progress.py	単発 Engine 実行の progress callback を整形する。	progress_reset(), emit_status_ progress(), format_progress(), progress_update(), progress_ message()	model worker 実行時

physics

階層1: 物理モデルを三層に分離した枝

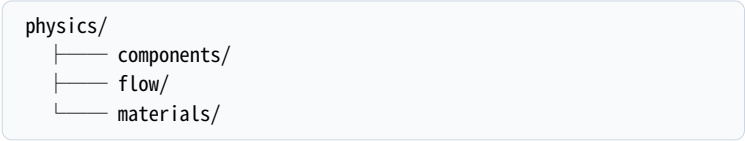
materials, flow, components に分け、材料物性・流れ構成式・実体部品を混ぜない。

読むもの	state and parameters from simulation	渡すもの	pressure, flow, thrust, derivatives
連携先	simulation.rhs	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/	feed_system, chamber, nozzle を外向き API として束ね、simulation から呼ばれる境界になる。
physics/flow/	オリフィス、二相 HEM/Dyer、PR 実在気体流、ノズル理想気体関係を置く。
physics/materials/	N2O など、部品をまたいで使う物性データと状態方程式を置く。



physics/materials

階層2: 材料物性

N2O など、部品をまたいで使う物性データと状態方程式を置く。

読むもの	temperature, pressure, molar volume, CSV data	渡すもの	thermodynamic properties
連携先	physics.flow, components	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/materials/  
└── nitrous_oxide/
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/materials/nitrous_oxide/	飽和表、PR-EOS、NASA9 理想気体物性を束ねる。

physics/materials/nitrous_oxide

階層3: N2O 物性の入口

飽和表、PR-EOS、NASA9 理想気体物性を束ねる。

読むもの	NIST CSV, PR constants, NASA9 coefficients	渡すもの	N2O property functions
連携先	feed_system tank/injector	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/materials/nitrous_oxide/  
├── eos/  
├── ideal_gas/  
└── saturation/
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/materials/nitrous_oxide/eos/	気相単独タンクと PR 気相オリフィスで使う実在気体圧力・departure 量を提供する。
physics/materials/nitrous_oxide/ideal_gas/	PR の理想気体基準として cp, h, s を一貫して計算する。
physics/materials/nitrous_oxide/saturation/	液相共存タンクと HEM で使う飽和圧、飽和体積、エネルギー、微分を提供する。

physics/materials/nitrous_oxide/saturation

階層4: NIST 飽和物性表

液相共存タンクと HEM で使う飽和圧、飽和体積、エネルギー、微分を提供する。

読むもの	NIST_Chemistry.csv, bounds policy	渡すもの	psat(T), h_l(T), u_v(T), derivatives
連携先	tank.liquid_phase, two_phase.hem	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割

physics/materials/nitrous_oxide/saturation/
├── access.py
├── bounds.py
├── paths.py
├── schema.py
└── table.py

名前	この階層での識別
access.py	bounds policy を適用した物性値・微分アクセスを提供する。
bounds.py	clip/raise など範囲外ポリシーを処理する。
paths.py	NIST CSV パスの解決を行う。
schema.py	NIST CSV に必要な列があるかを検査する。
table.py	NIST CSV から飽和表を読み、PCHIP 補間器を構築する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
access.py	bounds policy を適用した物性値・微分アクセスを提供する。	SaturationAccessMixin	物性評価時
bounds.py	clip/raise など範囲外ポリシーを処理する。	sanitize_coordinate(), ensure_finite()	物性評価時
paths.py	NIST CSV パスの解決を行う。	resolve_project_relative_path()	設定読み込み時
schema.py	NIST CSV に必要な列があるかを検査する。	validate_required_columns(), property_column_map(), csv_schema_info()	起動診断
table.py	NIST CSV から飽和表を読み、PCHIP 補間器を構築する。	N2OSaturationTable	Oxidizer 生成時

physics/materials/nitrous_oxide/eos

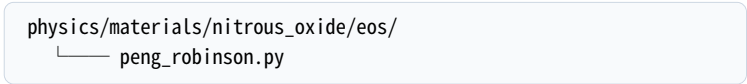
階層4: Peng-Robinson EOS

気相単独タンクと PR 気相オリフィスで使う実在気体圧力・departure 量を提供する。

読むもの	T, molar volume	渡すもの	p, u_dep, h_dep, s_dep, Cv corrections
連携先	gas_phase, real_gas.isentropic_orifice	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
peng_robinson.py	PR-EOS の圧力、departure 関数、実在気体 Cv 等を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
peng_robinson.py	PR-EOS の圧力、departure 関数、実在気体 Cv 等を計算する。	PengRobinsonEOS	気相タンク/PR 流量

physics/materials/nitrous_oxide/ideal_gas

階層4: NASA9 理想気体物性

PR の理想気体基準として cp, h, s を一貫して計算する。

読むもの	T and NASA9 coefficients	渡すもの	cp, cv, h, s
連携先	PR gas tank, PR gas orifice	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/materials/nitrous_oxide/ideal_gas/  
├── intervals.py  
├── nasa9.py  
└── polynomials.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
intervals.py	温度範囲ごとの NASA9 係数選択を行う。
nasa9.py	NASA9 係数を束ね、cp/cv/h/s を返す。
polynomials.py	NASA9 多項式の評価式を持つ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
intervals.py	温度範囲ごとの NASA9 係数選択を行う。	value_from_json_like(), normalize_coefficients(), select_interval()	NASA9 評価時
nasa9.py	NASA9 係数を束ね、cp/cv/h/s を返す。	NASA9IdealGas	PR 実在気体補正
polynomials.py	NASA9 多項式の評価式を持つ。	cp_over_R(), h_over_RT(), s_over_R()	NASA9Species 内部

physics/flow

階層2: 部品に依存しない流れ構成式

オリフィス、Valve の Kv 圧損、二相 HEM/Dyer、二相ボイド率診断、PR 実在気体流、ノズル理想気体関係を置く。

読むもの	local thermodynamic states and geometry	渡すもの	mass flow, mass flux, Mach relation, shock relation, diagnostic void fraction
連携先	components.feed_system, components.nozzle	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/flow/  
├── ideal_gas/  
├── orifice/  
├── real_gas/  
├── two_phase/  
└── valve/
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/flow/ideal_gas/	面積 Mach 数関係、等エントロピー比、垂直衝撃波関係を提供する。
physics/flow/orifice/	液相 SPI と気相 SPI で共通する Bernoulli 型流量式を提供する。液相 SPI は飽和液密度、気相 SPI はタンク気相密度を明示して呼び出す。
physics/flow/real_gas/	気相 N2O の圧縮性オリフィス流れを PR-EOS の等エントロピー線で扱う。
physics/flow/two_phase/	N2O のフラッシング、二相チョーク、ボイド率診断を扱う式を置く。
physics/flow/valve/	全開 Valve の Kv から液相圧損・必要 Kv を計算する。

physics/flow/orifice

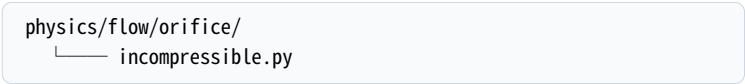
階層3: 非圧縮オリフィス式

液相 SPI と気相 SPI で共通する Bernoulli 型流量式を提供する。

読むもの	Cd, area, density, delta-p	渡すもの	mass flow
連携先	injector.liquid/gas	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
incompressible.py	$C_d \cdot A \cdot \sqrt{2 \cdot \rho \cdot \Delta p}$ の非圧縮オリフィス式を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
incompressible.py	$C_d \cdot A \cdot \sqrt{2 \cdot \rho \cdot \Delta p}$ の非圧縮オリフィス式を提供する。	incompressible_orifice_mass_flow()	SPI 流量

physics/flow/valve

階層3: 全開 Valve の Kv 圧損式

ユーザー入力の K_v を、水基準の全開 Valve 流量係数として扱い、液相 N2O の準定常圧損要素へ変換する。ここには部品状態や root solve は置かず、式だけを置く。

読むもの	K_v , liquid density, mass flow or delta-p	渡すもの	valve mass flow, pressure drop, required K_v
連携先	feed_system.valve, feed_system.coupling	代表フロー	K_v relation -> predictor/root -> FeedSystem

この枝の内部木構造

```
physics/flow/valve/  
├── __init__.py  
└── kv.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
kv.py	K_v と液相密度から $m_{\text{dot}} = K_v / 36000 \cdot \sqrt{\rho \cdot \Delta p}$ 、逆向きの圧損、必要 K_v を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
kv.py	全開 Valve の液相 K_v 圧損式と逆算式を提供する。	kv_liquid_mass_flow(), kv_liquid_pressure_drop(), kv_required_for_pressure_drop()	Valve coupling, sizing diagnostic

physics/flow/two_phase

階層3: 二相インジェクタ流量と診断

N2O のフラッシングと二相チョークを扱う HEM/Dyer の式、および Valve 出口の等エンタルピー品質・ボイド率診断を置く。

読むもの	saturation properties, pressure drop, quality, density, mass flux	渡すもの	HEM flux, Dyer blended flow, diagnostic void fraction
連携先	injector.liquid, valve.diagnostics	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/flow/two_phase/  
├── dyer/  
├── hem/  
└── void_fraction/
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/flow/two_phase/dyer/	SPI と HEM の間を上流条件に基づいて補間する。Valve 有効時は p_valve_exit をインジェクタ上流圧として使う。
physics/flow/two_phase/hem/	飽和線上の等エントロピー二相経路を評価し、質量流束最大点を探索する。
physics/flow/two_phase/void_fraction/	等エンタルピー品質、Homogeneous/Zivi/Rouhani-Axelsson 高流束近似のボイド率診断を提供する。

physics/flow/two_phase/void_fraction

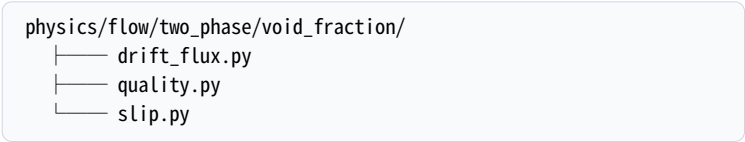
階層4: 品質・ボイド率診断

Valve 出口圧で平衡化した場合の品質と、複数のボイド率参考値を計算する。これらは診断値であり、デフォルトの第一段階 Valve モデルではインジェクタ入口状態として輸送しない。

読むもの	oxidizer saturation properties, quality, densities, mass flux, diameter	渡すもの	quality, void fraction diagnostics
連携先	feed_system.valve.diagnostics	代表フロー	isenthalpic flash -> slip/drift-flux models -> timeseries columns

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
quality.py	等エンタルピー平衡フラッシュ品質と homogeneous void fraction を計算する。
slip.py	Zivi slip ratio を使うボイド率を計算する。
drift_flux.py	Rouhani-Axelsson 高流束近似のボイド率を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
quality.py	Valve 出口圧での等エンタルピー品質と Homogeneous void fraction を作る。	equilibrium_quality_isenthalpic_flash(), homogeneous_void_fraction()	Valve diagnostics
slip.py	Zivi model の slip ratio と void fraction を作る。	zivi_slip_ratio(), zivi_void_fraction()	Valve diagnostics
drift_flux.py	高流束条件での Rouhani-Axelsson 近似 void fraction を作る。	rouhani_axelsson_high_flux_void_fraction()	Valve diagnostics

physics/flow/two_phase/hem

階層4: HEM 経路と臨界表

飽和線上の等エントロピー二相経路を評価し、質量流束最大点を探索する。

読むもの	upstream saturated liquid state	渡すもの	local two-phase state, critical table
連携先	injector.liquid.hem	代表フロー	upstream saturated state -> equilibrium path -> mass flux curve -> critical table

この枝の内部木構造

```
physics/flow/two_phase/hem/  
├── critical_table.py  
├── equilibrium_path.py  
└── states.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
critical_table.py	温度ごとの HEM 臨界流束テーブルを作成・補間する。
equilibrium_path.py	HEM の飽和線上等エントロピー経路を構築する。
states.py	HEM 局所状態と質量流束を表すデータ構造・計算を持つ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
critical_table.py	温度ごとの HEM 臨界流束テーブルを作成・補間する。	build_hem_critical_table(), interpolate_hem_critical_table()	HEM 高速化
equilibrium_path.py	HEM の飽和線上等エントロピー経路を構築する。	require_hem_properties(), saturated_expansion_curve()	HEM 局所状態評価
states.py	HEM 局所状態と質量流束を表すデータ構造・計算を持つ。	hem_local_state(), hem_critical_state_from_curve(), hem_critical_state_at_pressure(), select_hem_state(), argmax_mass_flux()	HEM 流量

physics/flow/two_phase/dyer

階層4: Dyer 補間

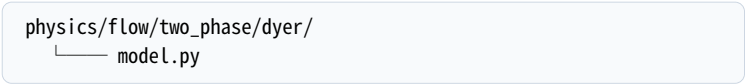
SPI と HEM の間を上流飽和条件に基づいて補間する。

読むもの	SPI flow, HEM flow, pressure ratio	渡すもの	Dyer blended flow, k
連携先	injector.liquid.dyer	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
model.py	Dyer 重み k と SPI/HEM ブレンドを計算する。



直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	Dyer 重み k と SPI/HEM ブレンドを計算する。	dyer_weight(), blend_spi_hem()	Dyer 流量

physics/flow/real_gas

階層3: 実在気体圧縮性流れ

気相 N2O の圧縮性オリフィス流れを PR-EOS の等エントロピー線で扱う。

読むもの	PR upstream state, back pressure	渡すもの	critical gas mass flux
連携先	injector.gas.pr	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

```
physics/flow/real_gas/
└── isentropic_orifice/
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/flow/real_gas/isentropic_orifice/	h0,s0 から局所状態を解き、G(p) 最大でチョークを判定する。

physics/flow/real_gas/isentropic_orifice

階層4: PR 気相等エントロピーオリフィス

h_0, s_0 から局所状態を解き、 $G(p)$ 最大でチョークを判定する。

読むもの	$T_0, v_0, p_{\text{back}}, \text{EOS}$	渡すもの	$\dot{m}$, selected state, choked flag
連携先	injector.gas.pr	代表フロー	upstream $h_0/s_0 \rightarrow$ local state $\rightarrow G(p) \rightarrow$ critical maximum

この枝の内部木構造

```
physics/flow/real_gas/isentropic_orifice/  
├── cache.py  
├── critical.py  
├── local_state.py  
└── upstream_state.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
cache.py	PR 気相流量の cache key と cache 管理を提供する。
critical.py	$G(p)$ 最大化でチョーク流束を探す。
local_state.py	指定圧力で $s=s_0$ となる下流局所状態を解く。
upstream_state.py	上流タンク気相から h_0, s_0, p_0 を作る。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
cache.py	PR 気相流量の cache key と cache 管理を提供する。	gas_cache_key()	PR 流量高速化
critical.py	$G(p)$ 最大化でチョーク流束を探す。	find_pr_gas_critical_state(), select_pr_gas_state()	PR 流量
local_state.py	指定圧力で $s=s_0$ となる下流局所状態を解く。	gas_temperature_bracket(), pr_ gas_isentropic_local_state()	$G(p)$ 評価
upstream_state.py	上流タンク気相から h_0, s_0, p_0 を作る。	pr_gas_upstream_state()	PR 流量開始時

physics/flow/ideal_gas

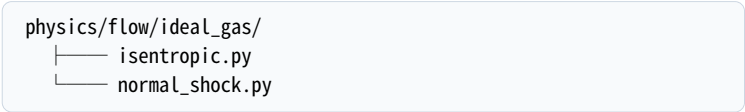
階層3: 理想気体ノズル関係

面積 Mach 数関係、等エントロピー比、垂直衝撃波関係を提供する。

読むもの	Mach number, gamma	渡すもの	pressure/temperature ratios, shock states
連携先	components.nozzle	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
isentropic.py	等エントロピー圧力比、温度比、area-Mach 関係を提供する。
normal_shock.py	垂直衝撃波前後の Mach/圧力比を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
isentropic.py	等エントロピー圧力比、温度比、area-Mach 関係を提供する。	area_ratio_from_mach(), pressure_ratio_from_mach(), temperature_ratio_from_mach(), shock_branch_mass_function(), choked_mass_function_factor()	Nozzle 分岐
normal_shock.py	垂直衝撃波前後の Mach/圧力比を計算する。	static_pressure_ratio_downstream_to_upstream(), exit_shock_pressure_ratio_to_stagnation()	Nozzle 内部衝撃波

physics/components

階層2: 実体部品モデル

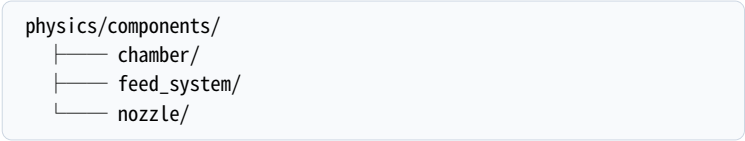
feed_system, chamber, nozzle を外向き API として束ね、simulation から呼ばれる境界になる。

読むもの	component parameters, state	渡すもの	flow results and derivatives
連携先	physics.flow, materials	代表フロー	components -> flow/materials -> numerics

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/chamber/	燃料後退、燃焼室内 storage、CEA 熱化学、圧力 ODE を外向き API にまとめる。
physics/components/feed_system/	Tank outlet state を Injector へ渡し、得た流量で Tank derivatives を計算する。
physics/components/nozzle/	燃焼室よどみ状態から分岐を判定し、質量流量・推力・性能指標を返す。



physics/components/feed_system

階層3: Tank, Valve, Injector を結合する供給系枝

Tank outlet state を、任意の Valve 圧損要素を通して Injector へ渡し、得た流量で Tank derivatives を計算する。simulation からは一つの FeedSystem facade として見える。

読むもの	tank state, chamber pressure, valve setting, injector params	渡すもの	n_dot oxidizer, tank derivatives, injector/valve details
連携先	simulation.rhs	代表フロー	tank pressure -> optional valve coupling -> injector mass flow -> tank derivatives

この枝の内部木構造

```
physics/components/feed_system/  
├── coupling/  
├── injector/  
├── tank/  
├── valve/  
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/feed_system/coupling/	Valve 有効時に、predictor または root で Valve 出口圧とインジェクタ流量を釣り合わせる。
physics/components/feed_system/injector/	液相 SPI/HEM/Dyer、気相 SPI/PR を選択し、酸化剤流量と診断値を返す。
physics/components/feed_system/tank/	液相共存タンク、気相単独タンク、初期化、出口物性をまとめる。
physics/components/feed_system/valve/ model.py	全開 Valve component と、Valve 列用の診断値を作る。 FeedSystem facade。Tank, optional Valve, Injector を結合し、simulation から見る供給系境界になる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	FeedSystem facade。Valve 無効時は Tank->Injector、有効液相時は Tank->Valve->Injector、気相時は Valve pass-through として扱う。	FeedSystem	RHS 評価

physics/components/feed_system/coupling

階層4: Valve/Injector 液相連成

Valve 出口圧 `p_valve_exit` を導入し、Valve を通る流量と Injector を通る流量を一致させる。ここは連成の制御を担い、Kv 式や Injector 式そのものは下位の flow/component へ委譲する。

読むもの	<code>p_tank</code> , <code>p_c</code> , Kv, liquid density, injector model	渡すもの	<code>p_valve_exit</code> , <code>m_dot</code> , residual diagnostics
連携先	<code>valve.model</code> , <code>injector.model</code> , <code>physics.flow.valve</code>	代表フロー	conductance predictor -> optional scalar root -> feed state

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
<code>predictor.py</code>	Valve と Injector を局所的な $m_{dot}=C*\sqrt{dp}$ と見て、反復なしで <code>p_valve_exit</code> を推定する。
<code>root.py</code>	predictor 残差が大きい場合に、Valve 流量と Injector 流量のスカラー root を解く。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
<code>predictor.py</code>	液相密度、Valve conductance、局所 Injector conductance から <code>p_valve_exit</code> を推定する。	<code>liquid_density()</code> , <code>valve_conductance()</code> , <code>conductance_predictor_pressure()</code> , <code>predict_valve_injector_state()</code>	Valve predictor/root
<code>root.py</code>	<code>m_dot_valve(p_valve_exit)-m_dot_injector(p_valve_exit)=0</code> を解く。	<code>solve_valve_injector_state()</code>	Valve root mode

physics/components/feed_system/valve

階層4: 全開 feed valve 部品

4_Valve.json の値を保持する component と、Valve 専用の診断値を作る枝である。液相圧損は固定仕様の Kv_SPI、気相は pass-through である。

読むもの	enabled, Kv, d_inner, coupling mode, void fraction model list	渡すもの	Valve object, details dict for Results columns, summary diagnostics
連携先	feed_system.coupling, results.timeseries.injector	代表フロー	Valve settings -> coupling -> diagnostics -> timeseries/summary

この枝の内部木構造

```
physics/components/feed_system/valve/  
├── diagnostics.py  
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
model.py	Valve の入力値、canonical mode、代表面積、root 診断カウンタを保持する。
diagnostics.py	Results シートの Valve 列と summary 用に、圧損、流量比、HEM/SPI 比、品質、ボイド率を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	全開 Valve component。enabled=false なら供給系は Valve なし、enabled=true なら液相のみ圧損要素として入る。	Valve, diagnostics(), record_root_result(), record_predictor_accept()	Engine 構築/RHS
diagnostics.py	p_valve_exit[MPa], dp_valve[MPa], G_HEM_G_SPI_ratio[\%], Void_Fraction_* などを作る。	valve_diagnostics()	timeseries 再構成

physics/components/feed_system/tank

階層4: N2O タンク部品

液相共存タンク、気相単独タンク、初期化、出口物性をまとめる。

読むもの	n_v, n_l, T_T, tank params	渡すもの	p_T, outlet state, d/dt tank states
連携先	materials.nitrous_oxide	代表フロー	tank outlet -> injector mass flow -> tank derivatives

この枝の内部木構造

```
physics/components/feed_system/tank/  
├── gas_phase/  
├── initialization/  
├── liquid_phase/  
├── outlet/  
├── oxidizer.py  
├── oxidizer_real_gas_api.py  
├── oxidizer_saturation_api.py  
├── oxidizer_saturation_core.py  
├── oxidizer_saturation_values.py  
├── oxidizer_tank_api.py  
└── parameters.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/feed_system/tank/gas_phase/	PR-EOS と開放系エネルギー式で気相ブローダウンを計算する。
physics/components/feed_system/tank/initialization/	総質量または充填率から n_v, n_l, T_T を決める。
physics/components/feed_system/tank/liquid_phase/	質量保存・体積拘束・エネルギー保存の3元連立を解く。
physics/components/feed_system/tank/outlet/	液相時は飽和液、気相時は PR 気相として入口状態を作る。
oxidizer.py	N2O 物性 facade。飽和表、PR-EOS、NASA9 API を束ねる。
oxidizer_real_gas_api.py	PR-EOS 気相 API を Oxidizer facade へ接続する。
oxidizer_saturation_api.py	飽和物性 API を合成する mixin。core と values をまとめる。
oxidizer_saturation_core.py	飽和表の bounds policy 同期など低レベル制御を担う。
oxidizer_saturation_values.py	p_sat, v_l, h_l など飽和値の forwarder を担う。
oxidizer_tank_api.py	タンク初期化やタンク状態向けの API を Oxidizer へ提供する。
parameters.py	タンク幾何・初期条件・下限温度などのパラメータを持つ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
oxidizer.py	N2O 物性 facade。飽和表、PR-EOS、NASA9 API を束ねる。	Oxidizer	Tank/Injector から
oxidizer_real_gas_api.py	PR-EOS 気相 API を Oxidizer facade へ接続する。	OxidizerRealGasAPIMixin	気相評価
oxidizer_saturation_api.py	飽和物性 API を合成する mixin。core と values をまとめる。	OxidizerSaturationAPIMixin	Oxidizer 内部
oxidizer_saturation_core.py	飽和表の bounds policy 同期など低レベル制御を担う。	OxidizerSaturationCoreMixin	Oxidizer 内部
oxidizer_saturation_values.py	p_sat, v_l, h_l など飽和値の forwarder を担う。	OxidizerSaturationValuesMixin	飽和物性アクセス

ファイル	責務	主な API	使われる時
oxidizer_tank_api.py	タンク初期化やタンク状態向けの API を Oxidizer へ提供する。	OxidizerTankAPIMixin	Tank 計算
parameters.py	タンク幾何・初期条件・下限温度などのパラメータを持つ。	Tank	Tank 生成

physics/components/feed_system/tank/initialization

階層5: タンク初期状態

総質量または充填率から n_v, n_l, T_T を決める。

読むもの	tank volume, fill rate or mass, saturation properties	渡すもの	initial tank state
連携先	config.factory	代表フロー	tank outlet -> injector mass flow -> tank derivatives

この枝の内部木構造

```
physics/components/feed_system/tank/initialization/  
└── state.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
state.py	タンク体積・充填率・総質量から初期 n_v, n_l を解く。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
state.py	タンク体積・充填率・総質量から初期 n_v, n_l を解く。	initial_temperature(), moles()	initial_- Tank 生成

physics/components/feed_system/tank/liquid_phase

階層5: 液相共存タンク ODE

質量保存・体積拘束・エネルギー保存の3元連立を解く。

読むもの	saturation properties, n_dot_out	渡すもの	dn_v, dn_l, dT/dt
連携先	simulation.rhs	代表フロー	tank outlet -> injector mass flow -> tank derivatives

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
derivatives.py	液相共存タンクの3元連立微分を解く。

直接ファイルの責務と主なAPI

ファイル	責務	主なAPI	使われる時
derivatives.py	液相共存タンクの3元連立微分を解く。	liquid_equilibrium_derivatives()	RHS liquid phase

physics/components/feed_system/tank/gas_phase

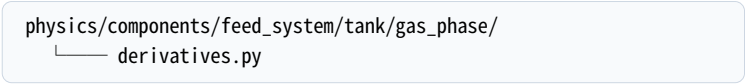
階層5: 気相単独タンク ODE

PR-EOS と開放系エネルギー式で気相ブローダウンを計算する。

読むもの	gas state, n_dot_out, PR/NASA9 properties	渡すもの	dn_v, dT/dt
連携先	simulation.rhs	代表フロー	tank outlet -> injector mass flow -> tank derivatives

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
derivatives.py	気相タンクの PR エネルギー式微分を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
derivatives.py	気相タンクの PR エネルギー式微分を計算する。	gas_pr_energy_derivatives()	RHS gas phase

physics/components/feed_system/tank/outlet

階層5: インジェクタ入口へ渡すタンク出口物性

液相時は飽和液、気相時は PR 気相として入口状態を作る。

読むもの	tank state, phase	渡すもの	outlet pressure, density, molar volume
連携先	injector.model	代表フロー	tank outlet -> injector mass flow -> tank derivatives

この枝の内部木構造

```
physics/components/feed_system/tank/outlet/  
└── properties.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
properties.py	インジェクタ入口へ渡す液相/気相出口物性を構築する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
properties.py	インジェクタ入口へ渡す液相/気相出口物性を構築する。	phase_is_liquid(), phase_mask(), tank_pressure(), orifice_molar_ volume()	Injector 呼び出し前

physics/components/feed_system/injector

階層4: インジェクタ部品

液相 SPI/HEM/Dyer、気相 SPI/PR を選択し、酸化剤流量と診断値を返す。インジェクタはタンクを直接知らず、FeedSystem から渡される上流圧を使う。

読むもの	upstream pressure, tank thermodynamic state, chamber pressure, injector params	渡すもの	n_dot, mdot, flow details
連携先	physics.flow, feed_system.coupling	代表フロー	phase selector -> liquid/gas adapter -> flow model -> details

この枝の内部木構造

```
physics/components/feed_system/injector/  
├── details/  
├── gas/  
├── liquid/  
├── diagnostics.py  
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/feed_system/injector/details/	計算本体から独立して、SPI/HEM/Dyer/PR の診断列を組み立てる。
physics/components/feed_system/injector/gas/	液相枯渇後の気相 SPI または PR 流量を component API として使う。
physics/components/feed_system/injector/liquid/	液相供給時の SPI, HEM, Dyer を component API として使う。Valve 有効時は p_valve_exit を上流圧として受ける。
diagnostics.py	インジェクタ計算回数や fallback などの診断を保持する。
model.py	Injector facade。相とモデルを選び、任意の上流圧で流量 API を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
diagnostics.py	インジェクタ計算回数や fallback などの診断を保持する。	InjectorDiagnosticsMixin	summary/timeseries
model.py	Injector facade。相、モデル、上流圧を受け取り流量 API を提供する。	Injector, mass_flow_from_inlet(), flow_details()	RHS 評価

Valve 有効時の Dyer。 第一段階モデルでは、Valve 出口で得られる仮想品質をインジェクタ入口状態として輸送しない。Dyer の SPI 側は p_valve_exit-p_c を使い、HEM 側は残り圧力区間に対する proxy upstream として評価する。品質とボイド率は Valve 診断値として出力する。

physics/components/feed_system/injector/liquid

階層5: 液相インジェクタ adapter

液相供給時の SPI, HEM, Dyer を component API として使う。

読むもの	saturated liquid inlet, p_chamber	渡すもの	liquid oxidizer flow
連携先	flow.orifice, flow.two_phase	代表フロー	phase selector -> liquid/gas adapter -> flow model -> details

この枝の内部木構造

```
physics/components/feed_system/injector/liquid/  
├── dyer.py  
├── hem.py  
└── spi.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
dyer.py	液相 Dyer 流量を SPI/HEM 結果から組み立てる adapter。
hem.py	液相 HEM 流量と臨界テーブルを component 側から扱う adapter。
spi.py	液相 SPI 流量を component 側から呼び出す adapter。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
dyer.py	液相 Dyer 流量を SPI/HEM 結果から組み立てる adapter。	DyerMixin	liquid model Dyer
hem.py	液相 HEM 流量と臨界テーブルを component 側から扱う adapter。	HemMixin	liquid model HEM
spi.py	液相 SPI 流量を component 側から呼び出す adapter。	LiquidSpiMixin	liquid model SPI

physics/components/feed_system/injector/gas

階層5: 気相インジェクタ adapter

液相枯渇後の気相 SPI または PR 流量を component API として使う。

読むもの	gas inlet state, p_chamber	渡すもの	gas oxidizer flow
連携先	flow.orifice, flow.real_gas	代表フロー	phase selector -> liquid/gas adapter -> flow model -> details

この枝の内部木構造

```
physics/components/feed_system/injector/gas/  
├── pr.py  
└── spi.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
pr.py	PR 実在気体圧縮性流量を component 側から呼び出す adapter。
spi.py	タンク気相密度とタンク圧を使う気相 SPI 流量 adapter。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
pr.py	PR 実在気体圧縮性流量を component 側から呼び出す adapter。	GasPrMixin	gas_blowdown_ model=PR
spi.py	タンク気相密度とタンク圧を使う気相 SPI 流量 adapter。	GasSpiMixin	gas_blowdown_ model=SPI

physics/components/feed_system/injector/details

階層5: 時系列出力用の流量詳細

計算本体から独立して、SPI/HEM/Dyer/PR の診断列を組み立てる。

読むもの	inlet state, selected model, cached flow values	渡すもの	details dict for timeseries
連携先	results.timeseries.injector	代表フロー	phase selector -> liquid/gas adapter -> flow model -> details

この枝の内部木構造

```
physics/components/feed_system/injector/details/  
├── base.py  
├── builder.py  
├── gas.py  
├── liquid_spi.py  
└── two_phase.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
base.py	flow_details 共通の圧力差・共通フィールドを作る。
builder.py	選択モデルに応じて details を組み立てる入口。
gas.py	気相 SPI/PR の詳細列を作る。
liquid_spi.py	液相 SPI の詳細列を作る。
two_phase.py	HEM/Dyer の詳細列、k や臨界状態を作る。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
base.py	flow_details 共通の圧力差・共通フィールドを作る。	base_result()	timeseries 後処理
builder.py	選択モデルに応じて details を組み立てる入口。	build_flow_details()	Injector.flow_details
gas.py	気相 SPI/PR の詳細列を作る。	gas_flow_details()	timeseries 後処理
liquid_spi.py	液相 SPI の詳細列を作る。	liquid_spi_details()	timeseries 後処理
two_phase.py	HEM/Dyer の詳細列、k や臨界状態を作る。	closed_two_phase_result(), hem_details(), dyer_details(), liquid_two_phase_details()	timeseries 後処理

details 枝は ODE 本体の流量計算ではなく、時系列出力に必要な診断列を再構成する枝である。計算結果を変えたいなら liquid/gas adapter を見る。出力列だけを増やしたいなら details を見る。

physics/components/chamber

階層3: 燃焼室部品

燃料後退、燃焼室内 storage、CEA 熱化学、圧力 ODE を外向き API にまとめる。

読むもの	r_port, m_o, m_f, p_C, m_dot_o	渡すもの	fuel flow, thermo state, dp_C/dt
連携先	simulation.rhs, nozzle	代表フロー	fuel_grain + storage + thermochemistry -> pressure ODE

この枝の内部木構造

```
physics/components/chamber/  
├── fuel_grain/  
├── pressure/  
├── storage/  
├── thermochemistry/  
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/chamber/fuel_grain/	ポート体積、燃料後退速度、燃料流入量、燃料残量を扱う。
physics/components/chamber/pressure/ physics/components/chamber/storage/	p_C を状態量として積分するため、理想気体式の微分形を計算する。 m_o, m_f から O/F を計算し、ノズル流出を酸化剤由来・燃料由来へ分配する。
physics/components/chamber/ thermochemistry/ model.py	RocketCEA direct、cached direct、thermo table を統合し、 T_C, W_C, gamma と偏微分を返す。 Chamber facade。fuel, storage, pressure, thermochemistry を束ねる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	Chamber facade。fuel, storage, pressure, thermochemistry を束ねる。	Chamber	RHS 評価

physics/components/chamber/fuel_grain

階層4: 燃料グレイン幾何と後退

ポート体積、燃料後退速度、燃料流入量、燃料残量を扱う。

読むもの	r_f, oxidizer flux, regression params	渡すもの	r_dot, m_dot_f, chamber volume
連携先	chamber.model	代表フロー	fuel_grain + storage + thermochemistry -> pressure ODE

この枝の内部木構造

```
physics/components/chamber/fuel_grain/  
├── model.py  
└── volume.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
model.py	燃料グレインの後退速度と燃料流量を計算する。
volume.py	ポート体積、燃焼室体積、体積微分を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	燃料グレインの後退速度と燃料流量を計算する。	Fuel	RHS 評価
volume.py	ポート体積、燃焼室体積、体積微分を計算する。	ChamberVolumeMixin	pressure ODE

physics/components/chamber/storage

階層4: 燃焼室内 storage

m_o, m_f から O/F を計算し、ノズル流出を酸化剤由来・燃料由来へ分配する。

読むもの	chamber stored masses, nozzle mdot	渡すもの	dm_o/dt, dm_f/dt, O/F
連携先	pressure.ode	代表フロー	fuel_grain + storage + thermochemistry -> pressure ODE

この枝の内部木構造

下位枝・直接ファイルの役割

```
physics/components/chamber/storage/  
├── mass_balance.py  
└── oxidizer.py
```

名前	この階層での識別
mass_balance.py	ノズル流出を燃料/酸化剤由来へ分配し storage 微分を作る。
oxidizer.py	燃焼室内酸化剤由来質量と O/F 計算の補助。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
mass_balance.py	ノズル流出を燃料/酸化剤由来へ分配し storage 微分を作る。	ChamberStorageMixin	RHS 評価
oxidizer.py	燃焼室内酸化剤由来質量と O/F 計算の補助。	Oxidizer	storage 評価

physics/components/chamber/pressure

階層4: 燃焼室圧力 ODE

p_C を状態量として積分するため、理想気体式の微分形を計算する。

読むもの	$m_C, V_C, \text{thermo partials}$	渡すもの	dp_C/dt
連携先	thermochemistry.model	代表フロー	fuel_grain + storage + thermochemistry -> pressure ODE

この枝の内部木構造

```
physics/components/chamber/pressure/  
└── ode.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
ode.py	CEA 偏微分を含む燃焼室圧力 ODE を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
ode.py	CEA 偏微分を含む燃焼室圧力 ODE を計算する。	ChamberPressureMixin	RHS 評価

physics/components/chamber/thermochemistry

階層4: 燃焼熱化学

RocketCEA direct、cached direct、thermo table を統合し、T_C,W_C,gamma と偏微分を返す。

読むもの	p_C, O/F, fuel card	渡すもの	ThermoState and partial derivatives
連携先	chamber.pressure, nozzle	代表フロー	query -> table or cached direct -> RocketCEA backend

この枝の内部木構造

```
physics/components/chamber/thermochemistry/  
├── cea/  
├── diagnostics/  
├── table/  
├── model.py  
└── query.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/chamber/thermochemistry/cea/	RocketCEA の単位系を SI 側から隠蔽し、直接計算を実行する。
physics/components/chamber/thermochemistry/diagnostics/	direct 計算と table 補間を比較し、範囲・誤差・fallback を検査する。
physics/components/chamber/thermochemistry/table/	CEA 計算を格子化・補間し、ODE 中の熱化学評価を高速化する。
model.py	Thermochemistry facade。table/direct/cached direct を選ぶ。
query.py	p, O/F のクリップ、table/direct query の選択を行う。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	Thermochemistry facade。table/direct/cached direct を選ぶ。	ChamberThermochemistryMixin	Chamber state_ from_p
query.py	p, O/F のクリップ、table/direct query の選択を行う。	ThermoQueryMixin	thermo evaluation

physics/components/chamber/thermochemistry/cea

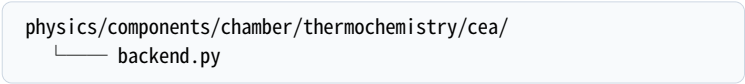
階層5: RocketCEA backend

RocketCEA の単位系を SI 側から隠蔽し、直接計算を実行する。

読むもの	p_C[Pa], O/F	渡すもの	T, W, gamma, cstar-like properties
連携先	table.builder, query	代表フロー	query -> table or cached direct -> RocketCEA backend

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
backend.py	RocketCEA 呼び出し、単位変換、direct cache を隠蔽する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
backend.py	RocketCEA 呼び出し、単位変換、direct cache を隠蔽する。	enter_new_fuel(), CEA	thermo direct

physics/components/chamber/thermochemistry/table

階層5: thermo table と cache

CEA 計算を格子化・補間し、ODE 中の熱化学評価を高速化する。

読むもの	grid settings, CEA backend, cache files	渡すもの	interpolated thermo state and derivatives
連携先	thermochemistry.model	代表フロー	signature -> cache io -> builder -> interpolator -> diagnostics

この枝の内部木構造

```
physics/components/chamber/thermochemistry/table/  
├── axis.py  
├── builder.py  
├── building.py  
├── compatibility.py  
├── diagnostic_cache.py  
├── initialization.py  
├── io.py  
├── management.py  
├── signature.py  
└── thermo_table.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
axis.py	圧力/O-F 軸の生成と範囲情報を持つ。
builder.py	RocketCEA backend を呼んで格子上の熱物性を計算する。
building.py	table 構築時の進行制御と例外処理をまとめる。
compatibility.py	cache signature や設定互換性を判定する。
diagnostic_cache.py	診断結果の再利用可否を判定・保存する。
initialization.py	Thermochemistry 初期化時に table/direct/cache を準備する。
io.py	npz table cache の読み書きを行う。
management.py	table cache, diagnostic, refinement の高レベル管理を行う。
signature.py	fuel card, grid, settings から cache signature を作る。
thermo_table.py	2D thermo table オブジェクト本体と補間アクセスを持つ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
axis.py	圧力/O-F 軸の生成と範囲情報を持つ。	make_axis()	table build
builder.py	RocketCEA backend を呼んで格子上の熱物性を計算する。	build_property_grids()	table build
building.py	table 構築時の進行制御と例外処理をまとめる。	build_thermo_table(), refined_limits()	table build
compatibility.py	cache signature や設定互換性を判定する。	compatible_with_table(), contains_point(), out_of_range_reason()	cache load
diagnostic_cache.py	診断結果の再利用可否を判定・保存する。	diagnostic_path(), write_table_diagnostic(), load_cached_table_diagnostic_if_valid()	diagnostic

ファイル	責務	主な API	使われる時
initialization.py	Thermochemistry 初期化時に table/direct/cache を準備する。	initialize_thermo_table_for(), _try_cached_table(), _diag- nose_cached_table(), _build_ or_refine_table(), _log_diagnos- tic_summary() ...	Thermochemistry 生 成
io.py	npz table cache の読み書きを行う。	save_table_npz(), load_table_ar- rays()	table cache
management.py	table cache, diagnostic, refinement の高レベル管理を行う。	ThermoTableManagementMixin	Thermochemistry 初 期化
signature.py	fuel card, grid, settings から cache signature を作る。	table_limits(), signature_pay- load(), payload_signature(), table_metadata(), metadata_ matches()	cache validation
thermo_table.py	2D thermo table オブジェクト本体と補間アクセスを持つ。	ThermoTable	thermo table query

table 枝は「値を返す表」だけではない。axis, signature, cache IO, builder, diagnostic cache, initialization がそれぞれ異なる責務を持つ。表の数値を変える変更と、cache 再利用条件を変える変更を混ぜないことが重要である。

physics/components/chamber/thermochemistry/diagnostics

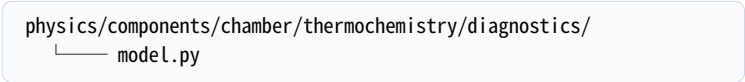
階層5: thermo table 診断

direct 計算と table 補間を比較し、範囲・誤差・fallback を検査する。

読むもの	table and direct samples	渡すもの	diagnostic summary
連携先	diagnostics.thermo_table	代表フロー	query -> table or cached direct -> RocketCEA backend

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
model.py	table と direct の比較診断を実行する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	table と direct の比較診断を実行する。	ThermoDiagnosticsMixin	diagnostic tool

physics/components/nozzle

階層3: ノズル部品

燃焼室よどみ状態から分岐を判定し、質量流量・推力・性能指標を返す。

読むもの	p_C, T_C, W_C, gamma, ambient pressure	渡すもの	mdot_n, thrust, c*, CF, Isp
連携先	flow.ideal_gas, simulation.rhs	代表フロー	thresholds -> branch state -> performance flow -> metrics/result

この枝の内部木構造

```
physics/components/nozzle/  
├── branch/  
├── performance/  
└── model.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
physics/components/nozzle/branch/	臨界圧、内部衝撃波、完全超音速などの枝を判定する。
physics/components/nozzle/performance/	流量、出口速度、推力、c*, CF, Isp の計算と結果辞書化を行う。
model.py	Nozzle facade。分岐判定と性能計算を統合して flow dict を返す。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model.py	Nozzle facade。分岐判定と性能計算を統合して flow dict を返す。	Nozzle	RHS 評価

physics/components/nozzle/branch

階層4: ノズル分岐判定

臨界圧、内部衝撃波、完全超音速などの枝を判定する。

読むもの	pressure ratio, expansion ratio, gamma	渡すもの	branch label and Mach states
連携先	nozzle.model	代表フロー	thresholds -> branch state -> performance flow -> metrics/result

この枝の内部木構造

```
physics/components/nozzle/branch/  
├── state.py  
└── thresholds.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
state.py	分岐結果を表す小さな状態構造を持つ。
thresholds.py	第一/第二臨界圧や臨界 Mach 数を計算する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
state.py	分岐結果を表す小さな状態構造を持つ。	nozzle_gas_state()	Nozzle.flow
thresholds.py	第一/第二臨界圧や臨界 Mach 数を計算する。	threshold_ratios()	Nozzle.flow

physics/components/nozzle/performance

階層4: ノズル性能量

流量、出口速度、推力、 c^* , CF, Isp の計算と結果辞書化を行う。

読むもの	branch state, chamber state, geometry	渡すもの	Nozzle flow result dict
連携先	results.timeseries.nozzle	代表フロー	thresholds -> branch state -> performance flow -> metrics/result

この枝の内部木構造

```
physics/components/nozzle/performance/  
├── flow.py  
├── geometry.py  
├── metrics.py  
└── result.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
flow.py	分岐ごとの出口状態、ノズル流量、推力を計算する。
geometry.py	スロート面積、出口面積、膨張比などを扱う。
metrics.py	c^* , CF, Isp など性能指標を計算する。
result.py	Nozzle.flow の結果辞書を組み立てる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
flow.py	分岐ごとの出口状態、ノズル流量、推力を計算する。	nozzle_mass_flow_from_state(), exit_velocity(), thrust(), cstar(), cf() ...	Nozzle.flow
geometry.py	スロート面積、出口面積、膨張比などを扱う。	throat_area_from_diameter_mm(), exit_area_from_expansion_ratio()	Nozzle 生成/flow
metrics.py	c^* , CF, Isp など性能指標を計算する。	specific_impulse_from_impulse()	Nozzle.flow
result.py	Nozzle.flow の結果辞書を組み立てる。	nozzle_flow_result()	timeseries/summary

results

階層1: 計算後データを作る枝

solution と event log から時系列表、summary、保存用データを作る。

読むもの	ODE solution, engine object	渡すもの	DataFrame, summary dict, persistence records
連携先	io	代表フロー	solution -> timeseries/summary -> persistence -> io

この枝の内部木構造

```
results/  
├── summary/  
├── timeseries/  
└── persistence.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
results/summary/	時系列と event log から最大値・停止理由に加え、全期間・液相主燃焼・気相尾部のインパルス、平均推力、Isp を分けて集約する。
results/timeseries/	solver が出した状態履歴を、タンク・インジェクタ・燃焼室・ノズルの列へ再評価する。
persistence.py	Engine の計算結果を timeseries, summary, event log として保存する流れをまとめる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
persistence.py	Engine の計算結果を timeseries, summary, event log として保存する流れをまとめる。	save_engine_results()	Engine.save_results

results/timeseries

階層2: 時系列再構成

solver が出した状態履歴を、タンク・Valve・インジェクタ・燃焼室・ノズルの列へ再評価する。時系列には t_step の基本サンプルと liquid_depleted などの event 時刻が含まれる。workbook のシート名は workbook_schema.py で定義される。

読むもの	solution, engine components	渡すもの	Results DataFrame, For_Mechanics DataFrame
連携先	io.excel.timeseries, io.excel.batch_summary	代表フロー	state samples -> tank/valve/injector/chamber/nozzle/mechanics-ready columns -> workbook

この枝の内部木構造

下位枝・直接ファイルの役割

```
results/timeseries/
├── builder.py
├── chamber.py
├── columns.py
├── injector.py
├── mechanics.py
├── nozzle.py
├── tank.py
└── workbook_schema.py
```

名前	この階層での識別
builder.py	solution を走査し、各時刻で component を再評価して Results 用 DataFrame を作る。
chamber.py	chamber 関連の時系列列を作る。
columns.py	Results シートの列順、カテゴリ、ヘッダー色を定義する。
injector.py	injector 関連の列と Valve 診断列を作る。
mechanics.py	飛翔・機構計算向けの質量、重心、慣性モーメント列を作る。
nozzle.py	nozzle/engineering performance 関連の時系列列を作る。
tank.py	tank 関連の時系列列を作る。
workbook_schema.py	Results と For_Mechanics のシート名定数を持つ。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
builder.py	solution を走査し、各時刻で component を再評価して Results 用 DataFrame を作る。	timeseries_category_palette(), ordered_timeseries_columns(), build_timeseries()	結果保存
columns.py	overall, tank, valve, injector, chamber, nozzle, performance の列順と色を定義する。	timeseries_category_palette(), timeseries_column_groups(), ordered_timeseries_columns()	Excel 出力
injector.py	injector 列と Valve 列を details dict から配列化する。	evaluate_injector_timeseries(), append_model_specific_columns()	timeseries builder

ファイル	責務	主な API	使われる時
mechanics.py	For_Mechanics シート用に、推力、推進剤質量、タンク/燃料の CG と慣性モーメントを作る。	build_mechanics_dataframe(), _tank_geometry(), _fuel_inertia()	Excel 出力
workbook_schema.py	timeseries workbook のシート名を単一ソースとして定義する。	RESULTS_SHEET_NAME, MECHANICS_SHEET_NAME	workbook 生成/monitor 抽出

event 時刻行。 Results と For_Mechanics は t_step の基本サンプルに加え、event log に記録された時刻を行として含む。相切替境界では event 処理後の状態を採用するため、液相枯渇行では tank_phase=gas、n_o_l[mol]=0 として再評価される。

Valve 列の配置。 Results シートでは overall->tank->valve->injector->chamber->nozzle->performance の順に列を置く。Valve 列は p_valve_exit[MPa], dp_valve[MPa], dp_valve_dp_total_ratio[\%], mdot_valve_mdot_valveless_ratio[\%], G_HEM_G_SPI_ratio[\%], quality_eq_valve_exit[-], Void_Fraction_HEM[-], Void_Fraction_Zivi[-], Void_Fraction_RA[-], G_valve[kg/m^2/s], u_valve[m/s], valve_mode である。気相 pass-through で定義できない圧損・流量比・流束比・G/u は空欄になる。

results/summary

階層2: summary 生成

時系列と event log から最大値・停止理由に加え、全期間・液相主燃焼・気相尾部のインパルス、平均推力、Isp、Valve 診断集約値を分けて集約する。

読むもの	timeseries, event log, component diagnostics	渡すもの	summary JSON/TXT data
連携先	batch.output, io.text	代表フロー	solution -> timeseries/summary -> persistence -> io

平均推力の扱い。 average_thrust_N は average_thrust_scope で示される範囲の平均推力である。液相主燃焼が存在する計算では average_thrust_scope=liquid_phase とし、average_thrust_N は liquid_phase_average_thrust_N と同じ値になる。全時刻を t_final_s まで含めた平均は all_phase_average_thrust_N に分離し、液相枯渇後の気相尾部は gas_phase_* の指標として出力する。

この枝の内部木構造

```
results/summary/  
└── builder.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
builder.py	時系列と event log から summary dict を作り、全期間・液相主燃焼・気相尾部の指標と Valve 液相診断を分離する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
builder.py	時系列と event log から summary dict を作り、全期間・液相主燃焼・気相尾部のインパルス、平均推力、Isp、Valve 診断を分離する。	build_summary(), _phase_summary_metrics(), _valve_summary_metrics()	save_results/batch

Valve summary。 Valve 有効時は、液相主燃焼区間の liquid_phase_avg_dp_valve_MPa, liquid_phase_max_dp_valve_MPa, liquid_phase_avg_dp_valve_dp_total_ratio_percent, liquid_phase_avg_mdot_valve_mdot_valveless_ratio_percent, liquid_phase_min_G_HEM_G_SPI_ratio_percent, liquid_phase_max_quality_eq_valve_exit, liquid_phase_max_Void_Fraction_*, valve_root_* などを出す。root 反復回数や残差は timeseries ではなく summary へ集約する。

io

階層1: ファイル I/O

Excel, JSON, text, filesystem 操作を物理モデルから切り離す。

読むもの	results/batch data structures	渡すもの	files and folders
連携先	app, batch, results	代表フロー	io -> parent branch

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
io/excel/	timeseries workbook と batch_summary workbook を作成する。 timeseries workbook は Results と For_Mechanics の 2 シート構成である。
io/filesystem/	出力フォルダ作成、設定コピー、ファイル名処理を担当する。
io/text/	summary や model sweep report を人間が読む txt として保存する。

```
io/
├── excel/
├── filesystem/
└── text/
```

io/text

階層2: text 出力

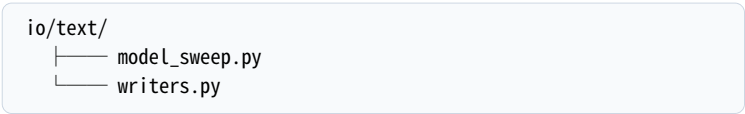
summary や model sweep report を人間が読む txt として保存する。

読むもの	summary dict, model summary	渡すもの	txt files
連携先	results, app.sweep	代表フロー	io/text -> parent branch

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
model_sweep.py	model_summary.txt/json を保存する。
writers.py	summary や event log を txt/json として保存する。



直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
model_sweep.py	model_summary.txt/json を保存する。	write_model_sweep_txt()	model sweep 終了時
writers.py	summary や event log を txt/json として保存する。	write_summary_txt(), event_log_txt()	write_結果保存

io/filesystem

階層2: filesystem 補助

出力フォルダ作成、設定コピー、ファイル名処理を担当する。

読むもの	paths	渡すもの	created folders, copied settings
連携先	app, batch, results	代表フロー	io/filesystem -> parent branch

この枝の内部木構造

```
io/filesystem/  
├── helpers.py  
└── settings_copy.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
helpers.py	フォルダ作成、safe filename、パス処理など。
settings_copy.py	実行時設定を Copy_Settings へコピーする。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
helpers.py	フォルダ作成、safe filename、パス処理など。	safe_filename_suffix(), copy_settings_to_result()	I/O 全域
settings_copy.py	実行時設定を Copy_Settings へコピーする。	copy_settings_to_common()	結果保存

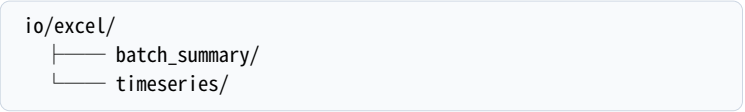
io/excel

階層2: Excel 出力

timeseries workbook と batch_summary workbook を作成する。timeseries workbook は Results と For_Mechanics の 2 シート構成である。

読むもの	DataFrame, batch rows, cell status	渡すもの	xlsx files
連携先	openpyxl	代表フロー	io/excel -> parent branch

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
io/excel/batch_summary/	Inputs/SPI/HEM/Dyer シート、ID/Status/診断列の色分けを行う。
io/excel/timeseries/	時系列データを Results シートへ、飛翔・機構向けデータを For_Mechanics シートへ保存する。

io/excel/timeseries

階層3: 時系列 Excel

時系列データを Results シートへ、飛翔・機構向けデータを For_Mechanics シートへ保存する。シート名は results/timeseries/workbook_schema.py の定数を使う。

読むもの	Results DataFrame, For_Mechanics DataFrame	渡すもの	timeseries_*.xlsx
連携先	results.timeseries	代表フロー	Results sheet + For_Mechanics sheet -> workbook

この枝の内部木構造

下位枝・直接ファイルの役割

名前	この階層での識別
writer.py	Results シートをカテゴリ色付きヘッダーで、For_Mechanics シートを外部連携向け列として書く。

```
io/excel/timeseries/
└── writer.py
```

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
writer.py	Results シートのカテゴリ色付きヘッダと For_Mechanics シートを持つ Excel へ書く。	style_timeseries_workbook(), write_timeseries_excel()	結果保存

For_Mechanics。 For_Mechanics は飛翔・機構計算へ渡すための補助シートである。先頭に t[s] と tank_phase を持つため、Monitored_Values.Source で For_Mechanics/<header> を指定した場合も average@liquid などの phase filter を利用できる。

- 常時出力する代表列: t[s], tank_phase, F[N], m_prop[kg], m_n_total[kg], tank_m_o[kg], unburned_fuel_mass[kg], m_o_chamber[kg], m_f_chamber[kg], m_chamber_storage[kg]。m_prop[kg] はタンク内酸化剤質量、未燃燃料グレイン質量、燃焼室内ガス貯留質量 m_o_chamber[kg]+m_f_chamber[kg] の和として扱う。
- 燃料列: 燃料を一様厚肉円筒として、CG_fuel[mm]=0, I_axis_fuel[kg*m^2], I_pitch_fuel[kg*m^2] を計算する。
- タンク列: 3_Tank/Tank_geometry/d が非 null なら、タンク内径とタンク体積から内側長さを導出し、円柱近似で CG_tank_even[mm], CG_tank_separated[mm], I_axis_tank[kg*m^2], I_pitch_tank_even[kg*m^2], I_pitch_tank_separated[kg*m^2] も追加する。

io/excel/batch_summary

階層3: batch_summary Excel

Inputs/SPI/HEM/Dyer シート、ID/Status/診断列の色分けを行う。Monitored_Values は timeseries workbook 内の指定シート・指定ヘッダーから完全一致で抽出する。

読むもの	batch summary rows, statuses, timeseries workbooks	渡すもの	batch_summary.xlsx
連携先	batch.output, results.timeseries	代表フロー	rows/status/monitoring/coloring/styles -> workbook

この枝の内部木構造

下位枝・直接ファイルの役割

```
io/excel/batch_summary/
├── coloring.py
├── monitoring.py
├── rows.py
├── status.py
├── styles.py
├── values.py
└── writer.py
```

名前	この階層での識別
coloring.py	status 判定を実際のセル塗りへ変換する。
monitoring.py	Monitored_Values.Source の Sheet/Header 指定から summary 値を抽出する。
rows.py	Inputs/SPI/HEM/Dyer 各シートの行を組み立てる。
status.py	Design ID/Status/Liquid_Depleted 等の総合判定を決める。
styles.py	batch_summary 用の薄緑/橙/赤/青/紫などの style を定義する。
values.py	Excel へ書く値の丸め、空欄、型整形を行う。
writer.py	batch_summary.xlsx 全体を組み立てる。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
coloring.py	status 判定を実際のセル塗りへ変換する。	apply_status_and_id_fills(), apply_model_status_cell_fills()	batch_summary 着色
monitoring.py	workbook の実シート・実ヘッダーから Monitored_Values を抽出する。	phase_mask(), extract_mon- itored_value(), locate_time- series_file()	batch_summary 書込
rows.py	Inputs/SPI/HEM/Dyer 各シートの行を組み立てる。	build_model_sheet_row()	batch_summary 書込
status.py	Design ID/Status/Liquid_Depleted 等の総合判定を決める。	is_success_status(), diagnostic_ cell_passes(), diagnostic_row_ passes(), design_has_error(), design_has_warning() ...	batch_summary 着色
styles.py	batch_summary 用の薄緑/橙/赤/青/紫などの style を定義する。	style_header_row(), autosize_ worksheet()	batch_summary 書込
values.py	Excel へ書く値の丸め、空欄、型整形を行う。	round_sig(), display_cell_value()	batch_summary 書込

ファイル	責務	主な API	使われる時
writer.py	batch_summary.xlsx 全体を組み立てる。	_write_inputs_sheet(), _write_model_sheet(), _write_batch_workbook()	batch 終了時

numerics

階層1: 物理非依存の数値道具

補間、root solve、最適化を物理モデルから切り離して提供する。

読むもの	scalar functions and arrays	渡すもの	interpolators, roots, maximizers
連携先	physics.materials, flow	代表フロー	physics call -> interpolation/root/optimization -> scalar result

この枝の内部木構造

```
numerics/  
├── interpolation/  
└── roots/
```

下位枝・直接ファイルの役割

名前	この階層での識別
numerics/interpolation/	NIST 飽和物性などで使う単調性を保ちやすい補間器を提供する。
numerics/roots/	Mach 数や圧力条件などの 1 変数方程式を安定に解く補助。

numerics/interpolation

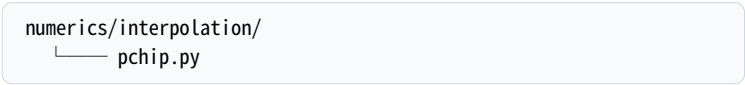
階層2: PCHIP 補間

NIST 飽和物性などで使う単調性を保ちやすい補間器を提供する。

読むもの	x-y data, log transform option	渡すもの	value and derivative
連携先	materials.saturation	代表フロー	physics call -> interpolation/root/optimization -> scalar result

この枝の内部木構造

下位枝・直接ファイルの役割



名前	この階層での識別
pchip.py	PCHIP 補間器と bounds/log transform 処理を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
pchip.py	PCHIP 補間器と bounds/log transform 処理を提供する。	Pchip1D	NIST saturation

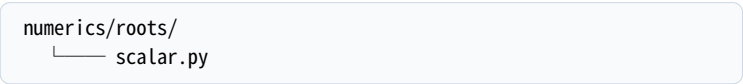
numerics/roots

階層2: スカラー方程式の root solve

Mach 数や圧力条件などの 1 変数方程式を安定に解く補助。

読むもの	scalar function and bracket	渡すもの	root value
連携先	nozzle, flow	代表フロー	physics call -> interpolation/root/optimization -> scalar result

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
scalar.py	bracket 付きスカラー root solve 補助を提供する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
scalar.py	bracket 付きスカラー root solve 補助を提供する。	root_toms_then_secant()	Mach/root solves

diagnostics

階層1: 診断処理

本計算ではなく、入力データ・表補間・比較グラフの検査を行う。

読むもの	results, cache, NIST CSV	渡すもの	reports, figures, diagnostics JSON
連携先	tools	代表フロー	tool entrypoint -> diagnostic runner -> report/figures

この枝の内部木構造

```
diagnostics/  
├── model_comparison/  
├── thermo_table/  
└── nist_schema.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
diagnostics/model_comparison/	モデル別時系列を集め、比較グラフを作る。
diagnostics/thermo_table/	table 補間と direct 計算の差を検査し、診断レポートを作る。
nist_schema.py	NIST CSV schema check を実行し、物理 component を生成せずに診断 JSON を作る。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
nist_schema.py	NIST CSV schema check を実行し、物理 component や RocketCEA を生成せずに診断 JSON を作る。	inspect_nist_csv()	run 開始時

diagnostics/model_comparison

階層2: SPI/HEM/Dyer 比較図

モデル別時系列を集め、比較グラフを作る。

読むもの	model result folders	渡すもの	comparison images
連携先	tools.generate_model_comparison_graphs	代表フロー	tool entrypoint -> diagnostic runner -> report/figures

この枝の内部木構造

下位枝・直接ファイルの役割

```
diagnostics/model_comparison/
├── data.py
├── image.py
├── plotting.py
└── specs.py
```

名前	この階層での識別
data.py	各モデルの timeseries を読み込み、plot_scope に応じて描画範囲を選択する。
image.py	グラフ画像保存の細部を扱う。
plotting.py	SPI/HEM/Dyer 比較グラフを描画する。
specs.py	比較グラフで読む列、ファイル名、軸ラベル、凡例位置、描画範囲、スタイル設定を定義する。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
data.py	各モデルの timeseries を読み込み、plot_scope に応じて描画範囲を選択する。	prefixed_filename(), locate_timeseries_file(), load_model_frames(), select_plot_data()	graph generation
image.py	グラフ画像保存の細部を扱う。	save_optimized_png()	graph generation
plotting.py	SPI/HEM/Dyer 比較グラフを描画する。	_draw_single_plot(), generate_model_comparison_graphs()	graph generation
specs.py	比較グラフで読む列、ファイル名、軸ラベル、凡例位置、描画範囲、スタイル設定を定義する。	-	graph generation

比較グラフ仕様。 diagnostics/model_comparison/specs.py の PLOT_SPECS は、各比較図について filename, column, ylabel を定義する。必要な図では legend_loc で凡例位置を指定でき、plot_scope で描画範囲を指定できる。plot_scope を省略した図は全時系列を描画し、liquid_only を指定した図は tank_phase 等に基づいて液相区間だけを描画する。質量流量、推力、圧力などの重要履歴は、仕様辞書で明示しない限り液相・気相を含む全領域を表示する。

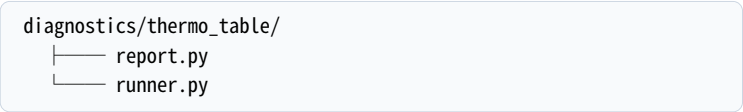
diagnostics/thermo_table

階層2: thermo table 診断 CLI 本体

table 補間と direct 計算の差を検査し、診断レポートを作る。

読むもの	settings and thermo table cache	渡すもの	diagnostic report
連携先	tools.run_thermo_table_diagnostic	代表フロー	tool entrypoint -> diagnostic runner -> report/figures

この枝の内部木構造



下位枝・直接ファイルの役割

名前	この階層での識別
report.py	thermo table 診断結果をテキスト/JSON に整える。
runner.py	thermo table 診断 CLI の実行本体。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
report.py	thermo table 診断結果をテキスト/JSON に整える。	relative_error(), safe_suffix(), write_summary_txt(), write_summary_json()	diagnostic tool
runner.py	thermo table 診断 CLI の実行本体。	_diagnostic_result_dir(), _write_unavailable_table_summary(), _validate_point(), _build_summary(), run_thermo_table_diagnostic()	tool entrypoint

tools

階層1: 直接実行できる補助 CLI

本体ロジックを diagnostics へ委譲し、ユーザーが単独実行しやすい入口だけを持つ。

読むもの	command line arguments	渡すもの	diagnostic artifacts
連携先	diagnostics	代表フロー	CLI arguments -> diagnostics modules

この枝の内部木構造

```
tools/  
├── generate_model_comparison_graphs.py  
└── run_thermo_table_diagnostic.py
```

下位枝・直接ファイルの役割

名前	この階層での識別
generate_model_comparison_graphs.py	比較グラフ生成の薄い CLI 入口。
run_thermo_table_diagnostic.py	thermo table 診断の薄い CLI 入口。

直接ファイルの責務と主な API

ファイル	責務	主な API	使われる時
generate_model_comparison_graphs.py	比較グラフ生成の薄い CLI 入口。	_find_project_root(), main()	手動実行
run_thermo_table_diagnostic.py	thermo table 診断の薄い CLI 入口。	_find_project_root(), main()	手動実行

変更対象から見る逆引き地図

変更したいこと	最初に見る枝	次に見る枝
通常実行と batch 実行の入口を変える	app/cli.py	app/model_sweep.py, batch/execution/executor.py
CSV 入力列や Design Name を変える	batch/input/	config/units/, batch/output/
一括実行中の表示を変える	batch/console/	batch/execution/progress_monitor.py
設定 JSON の検証を追加する	config/validation/	batch/input/row_config.py
N2O 飽和物性を変える	physics/materials/nitrous_oxide/saturation/	physics/components/feed_system/tank/
インジェクタ流量式を変える	physics/flow/	physics/components/feed_system/injector/
燃料後退式を変える	physics/components/chamber/fuel_grain/	physics/components/chamber/model.py
RocketCEA や thermo table を変える	physics/components/chamber/thermochemistry/	diagnostics/thermo_table/
ノズル分岐を変える	physics/components/nozzle/branch/	physics/flow/ideal_gas/
ODE 状態量や event を変える	simulation/state/, simulation/events/	simulation/rhs/, simulation/solver/
時系列列を増やす	results/timeseries/	io/excel/timeseries/
batch_summary の色を変える	io/excel/batch_summary/coloring.py	io/excel/batch_summary/status.py

全 Python ファイルの索引

この索引は、本文の枝ページを読んだ後にファイル単位で場所を確認するための一覧である。説明は重複を避け、各ファイルが他とどう違うかを識別できる粒度にしている。

ファイル	責務	主な API
app/cli.py	run.py から呼ばれる最上位 dispatcher。batch 検出と通常 sweep の分岐だけを担う。 通常実行のモデルスイープ全体を統括する。SPI/HEM/Dyer の結果を共通出力へまとめる。	main()
app/model_sweep.py		run_injector_model_sweep()
app/sweep/config.py	sweep 実行時のモデル選択、並列設定、thermo prewarm 条件を解釈する。	_set_json_value(), _dedupe_preserve_order(), _compact_none(), _prefixed_filename(), _planned_injector_models() ...
app/sweep/execution.py	逐次/並列に model worker を動かし、結果を回収する。	model_parallel_settings(), run_models_parallel(), run_models_sequential()
app/sweep/io.py	sweep 出力フォルダ、settings copy、summary 出力をまとめる。	Tee, model_sweep_output_root(), prepare_model_sweep_dirs(), write_sweep_common_outputs()
app/sweep/status.py	モデル worker の status JSON ペイロードと progress callback を作る。	_write_status(), _design_status_payload(), _make_design_status_callback()
app/sweep/worker.py	1 つのインジェクタモデルを実行する worker 単位。Engine の solve と保存を呼ぶ。	_run_single_model_worker(), _normalize_model_record()
batch/config/loader.py	Batch_Execution_Config.json を読み、CSV パスと Copy_Batch_Execution を扱う。	_read_json(), _batch_csv_basename(), detect_batch_execution(), _copy_batch_execution_dir()
batch/console/activity.py	準備フェーズだけに表示する activity のマイルストーンを決める。25/25 到達保証もここに 関わる。	preparation_activity_count(), activity_milestone_counts(), activity_milestone_lines()
batch/console/bars.py	activity 用のドットバー、progress 用のシャープバーなど文字バーの整形だけを担う。	format_progress_bar(), format_activity_bar()
batch/console/progress_units.py	設計内の SPI/HEM/Dyer 進捗を batch 全体の進捗単位へ換算する。	design_progress_units()
batch/console/status_io.py	worker status JSON を読み書きする。表示側と worker 側の共有ファイル境界。	write_status_file(), read_status_snapshot()
batch/constants.py	batch 内で共有する定数名、status 名などを持つ。	-

ファイル	責務	主な API
batch/execution/executor.py batch/execution/layout.py	batch 全体の高レベル入口。準備、実行、出力の各枝を順に呼ぶ。 設計 ID/モデル別の出力フォルダ構造を決める。	run_batch_execution() write_settings_dir_from_ config(), set_json_value(), prepare_worker_config(), ensure_design_layout()
batch/execution/parallel.py	複数設計を ProcessPool 等で並列実行する。	_parallel_failure_result(), _ run_parallel_designs()
batch/execution/preparation.py	batch 出力 root、入力 CSV、設計名、入力仕様など実行前準備をまとめる。	prepare_batch_result_ root(), print_batch_ header(), prewarm_or_ explain_thermo_cache()
batch/execution/progress_monitor.py	parallel/sequential 実行中に worker status を読んで console へ渡す。	progress_line_for_pend- ing()
batch/execution/results.py	worker から戻る成功/失敗結果の形を整える。	input_error_result(), sweep_result_to_row(), run_error_result()
batch/execution/sequential.py batch/execution/worker.py	設計を一つずつ順番に実行する。デバッグ時に有用。 1 設計ケースを実行する worker 本体。入力エラー時の扱いもここで分岐する。	_run_sequential_designs() _execute_design(), _exe- cute_design_worker()
batch/execution/worker_count.py batch/input/design_name.py	利用する worker 数を設計数・設定値から決める。 Input_Design_Name 列から設計名を読み、ファイル名安全な表記へ sanitize する。	_select_design_worker_ count() _sanitize_design_name(), _design_label(), _display_ design_label()
batch/input/destinations.py	Input_Specification の宛先 JSON パスを解決する。	_get_destination_node(), _get_destination_unit(), _set_destination_value(), _normalized_destination(), _columns_for_destina- tions()
batch/input/row_config.py	CSV 1 行を設定辞書へ反映し、InputError をセル位置へ戻す。	_prepare_row_config(), _infer_input_validation_re- jections()
batch/input/specs.py	Input_Specification, Input_Design_Name, Monitored_Values を読み取る。	_collect_input_specs(), _collect_design_name_col- umn(), _design_names_ from_csv(), _design_ labels_from_names(), _collect_monitor_specs()
batch/model.py	DesignCase や BatchResult など batch 用データ構造を定義する。	InputColumnSpec, Monitor- Spec, RowExecutionResult, BatchExecutionError

ファイル	責務	主な API
batch/output/artifacts.py	batch 全体の成果物保存をまとめて呼ぶ。	write_batch_outputs(), cleanup_worker_status_ files()
batch/output/model_rows.py	各モデル summary から batch_summary シート行を作る。	build_model_sheet_rows()
batch/output/reports.py	batch_errors.json/txt や input_mapping の保存を担当する。	_write_input_mapping_ text(), _write_design_ error_files(), _write_batch_ errors()
batch/planning/thermo.py	batch 入力が thermo signature を変え得るかを見て prewarm 可否を判断する。	_batch_overrides_ thermo_table_signature()
config/factory/accessors.py	settings dict から必須値や section を安全に取り出す。	unwrap_value(), require_ section(), require_value(), require_cstar_efficiency(), safe_str() ...
config/factory/builder.py	settings 読み込みから Engine 構築までの public 入口。	build_engine_from_con- fig(), build_engine_from_ settings()
config/factory/chamber.py	CEA, Fuel, ChamberOxidizer, Chamber を組み立てる。	build_chamber()
config/factory/engine.py	component bundle を Engine へ渡す最終構築。	build_engine_shell()
config/factory/feed_system.py	Oxidizer, Valve, Injector, FeedSystem を組み立てる。	build_feed_system()
config/factory/models.py	run_models や injector model 名の正規化を行う。	configured_run_models()
config/factory/nozzle.py	Nozzle 幾何と環境圧を構築する。	build_nozzle()
config/loading/project.py	プロジェクトルートと settings_dir の解決を行う。	find_project_root(), re- solve_settings_dir()
config/loading/settings.py	0_Settings 配下の JSON を読み込む。	read_json(), load_settings()
config/units/blank.py	空欄、null、未指定の扱いを一箇所に集約する。	is_blank()
config/units/conversion.py	実際の単位変換計算を行う。	conversion_spec(), con- vert_value()
config/units/converters.py	unit conversion の public facade。batch 側の import 面を短く保つ。	-
config/units/errors.py	単位変換エラー型とメッセージを扱う。	UnitConversionError
config/units/labels.py	単位ラベルの正規化と表示名を扱う。	extract_unit_from_label()
config/units/parser.py	Unit 文字列やセル表現を解析する。	_Parser, _dims_to_tuple(), _combine(), _power(), normalize_unit() ...
config/units/registry.py	対応する単位ラベルと SI 係数を登録する。	-
config/units/value.py	値と単位の組を扱う小さな構造を提供する。	ParsedUnit, Conversion- Spec, parse_feet_inches_ value()
config/validation/settings.py	幾何・充填率・効率・モデル選択と、Value 並列キー/Options などの settings 書式を検証する。	validate_config_values()

ファイル	責務	主な API
diagnostics/model_comparison/data.py	各モデルの timeseries を読み込む。	prefixed_filename(), locate_timeseries_file(), load_model_frames(), select_plot_data()
diagnostics/model_comparison/image.py	グラフ画像保存の細部を扱う。	save_optimized_png()
diagnostics/model_comparison/plotting.py	SPI/HEM/Dyer 比較グラフを描画する。	_draw_single_plot(), generate_model_comparison_graphs()
diagnostics/model_comparison/specs.py	比較グラフで読む列、ファイル名、軸ラベル、凡例位置、描画範囲、スタイル設定を定義する。	-
diagnostics/nist_schema.py	NIST CSV schema check を実行し、物理 component や RocketCEA を生成せずに診断 JSON を作る。	inspect_nist_csv()
diagnostics/thermo_table/report.py	thermo table 診断結果をテキスト/JSON に整える。	relative_error(), safe_suffix(), write_summary_txt(), write_summary_json()
diagnostics/thermo_table/runner.py	thermo table 診断 CLI の実行本体。	_diagnostic_result_dir(), _write_unavailable_table_summary(), _validate_point(), _build_summary(), run_thermo_table_diagnostic()
io/excel/batch_summary/coloring.py	status 判定を実際のセル塗りへ変換する。	apply_status_and_id_fills(), apply_model_status_cell_fills()
io/excel/batch_summary/monitoring.py	Monitored_Values の Sheet/Header 指定に従い、生成済み timeseries workbook から summary 値を抽出する。	phase_mask(), extract_monitored_value(), locate_timeseries_file()
io/excel/batch_summary/rows.py	Inputs/SPI/HEM/Dyer 各シートの行を組み立てる。	build_model_sheet_row()
io/excel/batch_summary/status.py	Design ID/Status/Liquid_Depleted 等の総合判定を決める。	is_success_status(), diagnostic_cell_passes(), diagnostic_row_passes(), design_has_error(), design_has_warning() ...
io/excel/batch_summary/styles.py	batch_summary 用の薄緑/橙/赤/青/紫などの style を定義する。	style_header_row(), auto_size_worksheet()
io/excel/batch_summary/values.py	Excel へ書く値の丸め、空欄、型整形を行う。	round_sig(), display_cell_value()
io/excel/batch_summary/writer.py	batch_summary.xlsx 全体を組み立てる。	_write_inputs_sheet(), _write_model_sheet(), _write_batch_workbook()
io/excel/timeseries/writer.py	時系列 DataFrame をカテゴリ色付き Excel へ書く。	style_timeseries_workbook(), write_timeseries_excel()

ファイル	責務	主な API
io/filesystem/helpers.py	フォルダ作成、safe filename、パス処理など。	safe_filename_suffix(), copy_settings_to_result()
io/filesystem/settings_copy.py	実行時設定を Copy_Settings へコピーする。	copy_settings_to_com- mon()
io/text/model_sweep.py	model_summary.txt/json を保存する。	write_model_sweep_txt()
io/text/writers.py	summary や event log を txt/json として保存する。	write_summary_txt(), write_event_log_txt()
numerics/interpolation/pchip.py	PCHIP 補間器と bounds/log transform 処理を提供する。	Pchip1D
numerics/roots/scalar.py	bracket 付きスカラー root solve 補助を提供する。	root_toms_then_secant()
physics/components/chamber/fuel_grain/model.py	燃料グレインの後退速度と燃料流量を計算する。	Fuel
physics/components/chamber/fuel_grain/volume.py	ポート体積、燃焼室体積、体積微分を計算する。	ChamberVolumeMixin
physics/components/chamber/model.py	Chamber facade。fuel, storage, pressure, thermochemistry を束ねる。	Chamber
physics/components/chamber/pressure/ode.py	CEA 偏微分を含む燃焼室圧力 ODE を計算する。	ChamberPressureMixin
physics/components/chamber/storage/mass_balance.py	ノズル流出を燃料/酸化剤由来へ分配し storage 微分を作る。	ChamberStorageMixin
physics/components/chamber/storage/oxidizer.py	燃焼室内酸化剤由来質量と O/F 計算の補助。	Oxidizer
physics/components/chamber/thermochemistry/cea/backend.py	RocketCEA 呼び出し、単位変換、direct cache を隠蔽する。	enter_new_fuel(), CEA
physics/components/chamber/thermochemistry/diagnostics/model.py	table と direct の比較診断を実行する。	ThermoDiagnosticsMixin
physics/components/chamber/thermochemistry/model.py	Thermochemistry facade。table/direct/cached direct を選ぶ。	ChamberThermochemistryMixin
physics/components/chamber/thermochemistry/query.py	p, O/F のクリップ、table/direct query の選択を行う。	ThermoQueryMixin
physics/components/chamber/thermochemistry/table/axis.py	圧力/O-F 軸の生成と範囲情報を持つ。	make_axis()
physics/components/chamber/thermochemistry/table/builder.py	RocketCEA backend を呼んで格子上の熱物性を計算する。	build_property_grids()
physics/components/chamber/thermochemistry/table/building.py	table 構築時の進行制御と例外処理をまとめる。	build_thermo_table(), refined_limits()
physics/components/chamber/thermochemistry/table/compatibility.py	cache signature や設定互換性を判定する。	compatible_with_table(), contains_point(), out_of_- range_reason()
physics/components/chamber/thermochemistry/table/diagnostic_cache.py	診断結果の再利用可否を判定・保存する。	diagnostic_path(), write_- table_diagnostic(), load_- cached_table_diagnostic_- if_valid()
physics/components/chamber/thermochemistry/table/initialization.py	Thermochemistry 初期化時に table/direct/cache を準備する。	initialize_thermo_table_- for(), _try_cached_table(), _diagnose_cached_table(), _build_or_refine_table(), _- log_diagnostic_summary() ...
physics/components/chamber/thermochemistry/table/io.py	npz table cache の読み書きを行う。	save_table_npz(), load_- table_arrays()
physics/components/chamber/thermochemistry/table/management.py	table cache, diagnostic, refinement の高レベル管理を行う。	ThermoTableManagementMixin

ファイル	責務	主な API
physics/components/chamber/thermochemistry/table/signature.py	fuel card, grid, settings から cache signature を作る。	table_limits(), signature_payload(), payload_signature(), table_metadata(), metadata_matches()
physics/components/chamber/thermochemistry/table/thermo_table.py	2D thermo table オブジェクト本体と補間アクセスを持つ。	ThermoTable
physics/components/feed_system/injector/details/base.py	flow_details 共通の圧力差・共通フィールドを作る。	base_result()
physics/components/feed_system/injector/details/builder.py	選択モデルに応じて details を組み立てる入口。	build_flow_details()
physics/components/feed_system/injector/details/gas.py	気相 SPI/PR の詳細列を作る。	gas_flow_details()
physics/components/feed_system/injector/details/liquid_spi.py	液相 SPI の詳細列を作る。	liquid_spi_details()
physics/components/feed_system/injector/details/two_phase.py	HEM/Dyer の詳細列、k や臨界状態を作る。	closed_two_phase_result(), hem_details(), dyer_details(), liquid_two_phase_details()
physics/components/feed_system/injector/diagnostics.py	インジェクタ計算回数や fallback などの診断を保持する。	InjectorDiagnosticsMixin
physics/components/feed_system/injector/gas/pr.py	PR 実在気体圧縮性流量を component 側から呼び出す adapter。	GasPrMixin
physics/components/feed_system/injector/gas/spi.py	タンク気相密度とタンク圧を使う気相 SPI 流量 adapter。	GasSpiMixin
physics/components/feed_system/injector/liquid/dyer.py	液相 Dyer 流量を SPI/HEM 結果から組み立てる adapter。	DyerMixin
physics/components/feed_system/injector/liquid/hem.py	液相 HEM 流量と臨界テーブルを component 側から扱う adapter。	HemMixin
physics/components/feed_system/injector/liquid/spi.py	液相 SPI 流量を component 側から呼び出す adapter。	LiquidSpiMixin
physics/components/feed_system/injector/model.py	Injector facade。相、モデル、任意の上流圧を受け取り流量 API を提供する。	Injector
physics/components/feed_system/model.py	Tank, optional Valve, Injector を結合する FeedSystem facade。simulation から見る供給系境界。	FeedSystem
physics/components/feed_system/coupling/predictor.py	Valve と Injector を局所 conductance で近似し、p_valve_exit を非反復で推定する。	predict_valve_injector_state()
physics/components/feed_system/coupling/root.py	Valve 流量と Injector 流量のスカラー root を解き、必要に応じて predictor 結果から bracket を縮める。	solve_valve_injector_state()
physics/components/feed_system/valve/model.py	全開 Valve component。Kv, d_inner, coupling mode, root 診断カウンタを保持する。	Valve
physics/components/feed_system/valve/diagnostics.py	Valve 列用の圧損、流量比、HEM/SPI 比、品質、ボイド率を計算する。	valve_diagnostics()
physics/components/feed_system/tank/gas_phase/derivatives.py	気相タンクの PR エネルギー式微分を計算する。	gas_pr_energy_derivatives()
physics/components/feed_system/tank/initialization/state.py	タンク体積・充填率・総質量から初期 n_v, n_l を解く。	initial_temperature(), initial_moles()
physics/components/feed_system/tank/liquid_phase/derivatives.py	液相共存タンクの 3 元連立微分を解く。	liquid_equilibrium_derivatives()
physics/components/feed_system/tank/outlet/properties.py	インジェクタ入口へ渡す液相/気相出口物性を構築する。	phase_is_liquid(), phase_mask(), tank_pressure(), orifice_molar_volume()
physics/components/feed_system/tank/oxidizer.py	N2O 物性 facade。飽和表、PR-EOS、NASA9 API を束ねる。	Oxidizer
physics/components/feed_system/tank/oxidizer_real_gas_api.py	PR-EOS 気相 API を Oxidizer facade へ接続する。	OxidizerRealGasAPIMixin
physics/components/feed_system/tank/oxidizer_saturation_api.py	飽和物性 API を合成する mixin。core と values をまとめる。	OxidizerSaturationAPIMixin

ファイル	責務	主な API
physics/components/feed_system/tank/oxidizer_saturation_core.py	飽和表の bounds policy 同期など低レベル制御を担う。	OxidizerSaturationCoreMixin
physics/components/feed_system/tank/oxidizer_saturation_values.py	p_sat, v_l, h_l など飽和値の forwarder を担う。	OxidizerSaturationValuesMixin
physics/components/feed_system/tank/oxidizer_tank_api.py	タンク初期化やタンク状態向けの API を Oxidizer へ提供する。	OxidizerTankAPIMixin
physics/components/feed_system/tank/parameters.py	タンク幾何・初期条件・下限温度などのパラメータを持つ。	Tank
physics/components/nozzle/branch/state.py	分岐結果を表す小さな状態構造を持つ。	nozzle_gas_state()
physics/components/nozzle/branch/thresholds.py	第一/第二臨界圧や臨界 Mach 数を計算する。	threshold_ratios()
physics/components/nozzle/model.py	Nozzle facade。分岐判定と性能計算を統合して flow dict を返す。	Nozzle
physics/components/nozzle/performance/flow.py	分岐ごとの出口状態、ノズル流量、推力を計算する。	nozzle_mass_flow_from_-state(), exit_velocity(), thrust(), cstar(), cf() ...
physics/components/nozzle/performance/geometry.py	スロート面積、出口面積、膨張比などを扱う。	throat_area_from_diameter_mm(), exit_area_from_expansion_ratio()
physics/components/nozzle/performance/metrics.py	c*, CF, lsp など性能指標を計算する。	specific_impulse_from_impulse()
physics/components/nozzle/performance/result.py	Nozzle.flow の結果辞書を組み立てる。	nozzle_flow_result()
physics/flow/ideal_gas/isentropic.py	等エントロピー圧力比、温度比、area-Mach 関係を提供する。	area_ratio_from_mach(), pressure_ratio_from_mach(), temperature_ratio_from_mach(), shock_branch_mass_function(), choked_mass_function_factor()
physics/flow/ideal_gas/normal_shock.py	垂直衝撃波前後の Mach/圧力比を計算する。	static_pressure_ratio_downstream_to_upstream(), exit_shock_pressure_ratio_to_stagnation()
physics/flow/orifice/incompressible.py	$C_d \cdot A \cdot \sqrt{2 \cdot \rho \cdot \Delta p}$ の非圧縮オリフィス式を提供する。	incompressible_orifice_mass_flow()
physics/flow/valve/kv.py	全開 Valve の Kv 液相圧損式、逆算式、必要 Kv 式を提供する。	kv_liquid_mass_flow(), kv_liquid_pressure_drop(), kv_required_for_pressure_drop()
physics/flow/real_gas/isentropic_orifice/cache.py	PR 気相流量の cache key と cache 管理を提供する。	gas_cache_key()
physics/flow/real_gas/isentropic_orifice/critical.py	G(p) 最大化でチョーク流束を探す。	find_pr_gas_critical_state(), select_pr_gas_state()
physics/flow/real_gas/isentropic_orifice/local_state.py	指定圧力で s=s0 となる下流局所状態を解く。	gas_temperature_bracket(), pr_gas_isentropic_local_state()
physics/flow/real_gas/isentropic_orifice/upstream_state.py	上流タンク気相から h0,s0,p0 を作る。	pr_gas_upstream_state()

ファイル	責務	主な API
physics/flow/two_phase/dyer/model.py	Dyer 重み k と SPI/HEM ブレンドを計算する。	dyer_weight(), blend_spi_hem()
physics/flow/two_phase/hem/critical_table.py	温度ごとの HEM 臨界流束テーブルを作成・補間する。	build_hem_critical_table(), interpolate_hem_critical_table()
physics/flow/two_phase/hem/equilibrium_path.py	HEM の飽和線上等エントロピー経路を構築する。	require_hem_properties(), saturated_expansion_curve()
physics/flow/two_phase/hem/states.py	HEM 局所状態と質量流束を表すデータ構造・計算を持つ。	hem_local_state(), hem_critical_state_from_curve(), hem_critical_state_at_pressure(), select_hem_state(), argmax_mass_flux()
physics/flow/two_phase/void_fraction/quality.py	等エンタルピー品質と Homogeneous void fraction を計算する。	equilibrium_quality_isenthalpic_flash(), homogeneous_void_fraction()
physics/flow/two_phase/void_fraction/slip.py	Zivi slip ratio と Zivi void fraction を計算する。	zivi_slip_ratio(), zivi_void_fraction()
physics/flow/two_phase/void_fraction/drift_flux.py	Rouhani-Axelsson 高流束近似の void fraction を計算する。	rouhani_axelsson_high_flux_void_fraction()
physics/materials/nitrous_oxide/eos/peng_robinson.py	PR-EOS の圧力、departure 関数、実在気体 C_v 等を計算する。	PengRobinsonEOS
physics/materials/nitrous_oxide/ideal_gas/intervals.py	温度範囲ごとの NASA9 係数選択を行う。	value_from_json_like(), normalize_coefficients(), select_interval()
physics/materials/nitrous_oxide/ideal_gas/nasa9.py	NASA9 係数を束ね、 $cp/cv/h/s$ を返す。	NASA9IdealGas
physics/materials/nitrous_oxide/ideal_gas/polynomials.py	NASA9 多項式の評価式を持つ。	cp_over_R(), h_over_RT(), s_over_R()
physics/materials/nitrous_oxide/saturation/access.py	bounds policy を適用した物性値・微分アクセスを提供する。	SaturationAccessMixin
physics/materials/nitrous_oxide/saturation/bounds.py	clip/raise など範囲外ポリシーを処理する。	sanitize_coordinate(), ensure_finite()
physics/materials/nitrous_oxide/saturation/paths.py	NIST CSV パスの解決を行う。	resolve_project_relative_path()
physics/materials/nitrous_oxide/saturation/schema.py	NIST CSV に必要な列があるかを検査する。	validate_required_columns(), property_column_map(), csv_schema_info()
physics/materials/nitrous_oxide/saturation/table.py	NIST CSV から飽和表を読み、PCHIP 補間器を構築する。	N2OSaturationTable
results/persistence.py	Engine の計算結果を timeseries, summary, event log として保存する流れをまとめる。	save_engine_results()
results/summary/builder.py	時系列と event log から summary dict を作り、全期間・液相主燃焼・気相尾部のインパルス、平均推力、 I_{sp} を分離する。	build_summary(), _phase_summary_metrics()

ファイル	責務	主な API
results/timeseries/builder.py	solution を走査し、各時刻で component を再評価して Results 用 DataFrame を作る。	timeseries_category_ palette(), ordered_time- series_columns(), build_ timeseries()
results/timeseries/chamber.py	chamber 関連の時系列列を作る。	evaluate_chamber_time- series(), evaluate_storage_ rates()
results/timeseries/columns.py	Results シートの列順、カテゴリ、ヘッダー色を定義する。Valve 列もここで配置される。	timeseries_category_ palette(), timeseries_col- umn_groups(), ordered_ timeseries_columns()
results/timeseries/injector.py	injector 関連列と Valve 診断列を作る。	_array(), evaluate_injec- tor_timeseries(), active_in- jector_model(), append_ model_specific_columns()
results/timeseries/mechanics.py	For_Mechanics シート用に phase、推力、質量、重心、慣性モーメントを作る。タンク内径からタンク長さを導出する。	build_mechanics_ dataframe()
results/timeseries/nozzle.py	nozzle/engineering performance 関連の時系列列を作る。	evaluate_nozzle_time- series()
results/timeseries/tank.py	tank 関連の時系列列を作る。	evaluate_tank_timeseries()
results/timeseries/workbook_schema.py	timeseries workbook のシート名を単一ソースとして定義する。	RESULTS_SHEET_NAME, MECHANICS_SHEET_NAME
simulation/events/engine_events.py	液相枯渇、温度下限、推力カットオフ等の event 関数を提供する。	current_phase(), thrust_ margin(), liquid_remain- ing(), tank_temperature_ margin_event(), fuel_port_ margin_event() ...
simulation/progress/engine_progress.py	単発 Engine 実行の progress callback を整形する。	progress_reset(), emit_sta- tus_progress(), format_ progress(), progress_up- date(), progress_message()
simulation/rhs/derivatives.py	StepEvaluation を状態微分 dy/dt へ変換する。	engine_derivative()
simulation/rhs/evaluation.py	一時刻に必要な中間量を評価し、重複計算を防ぐ。	chamber_nozzle_flow()
simulation/solver/engine_runner.py	solve_ivp segment loop 本体。event 発生ごとに状態を整える。	solve_engine()
simulation/solver/event_handling.py	event 後の phase 切替、clamp、停止判定を整理する。	handle_segment_event()
simulation/solver/segments.py	segment ごとの解を結合し、event 境界時刻では後続 segment の状態を優先して評価する。	triggered_event_name(), termination_category(), evaluate_segments()

ファイル	責務	主な API
simulation/solver/state.py	solver 状態、統計、診断カウンタ初期化、event 時刻を含む最終 solution の組み立てを担当する。	reset_engine_solve_state(), prepare_initial_segment_ state(), accumulate_ solver_stats(), finalize_en- gine_solution()
simulation/state/indices.py	状態ベクトル配列の添字を一元管理する。	-
simulation/state/initialization.py	各 component から ODE 初期状態 y0 を作る。	initial_engine_state()
simulation/state/jacobian.py	Radau 用の Jacobian sparsity 構造を作る。	build_engine_jac_sparsity()
simulation/system/engine.py	外部から見る Engine facade。solve や保存を下位モジュールへ委譲する。	Engine
tools/generate_model_comparison_graphs.py	比較グラフ生成の薄い CLI 入口。	_find_project_root(), main()
tools/run_thermo_table_diagnostic.py	thermo table 診断の薄い CLI 入口。	_find_project_root(), main()